

Steven Michael Garbeil

SSID: 79127

COINS Senior Portfolio

Senior Project Advisor: Valerie Sessions, Ph.d

April 1, 2004





Syllabus

Computer and Information Sciences (COINS)

COIN 496 Senior Project Portfolio Review

Course Description: This course is a prerequisite for all Senior Project courses – COIN 497, COIN 498, COIN 499, and ensures that the student has adequate mastery of the fundamentals of Computer Science in order to begin designing a Senior Project. Students must submit a portfolio reviewing their work in other upper level classes. This portfolio is reviewed by the Chair of the COINS department, as well as other COINS professors, and determines whether the student has adequate knowledge to advance farther. This course is Pass/Fail.

Course Objectives: At the end of this course, the student is expected to create a three-ring binder portfolio that includes the following:

- Title Page with Student Name, COIN 496 Senior Portfolio, and date.
- One program (source code and executable application) from each of the following courses: COIN 315, COIN 325, COIN 332, COIN 334. Each program must be loaded onto its own CD/DVD medium and labeled with the Student Name, Program Name, and COIN class for which it was submitted. The program should be well commented throughout and contain a minimum of the following as a header comment:
 - Student Name:
 - Program Name:
 - Creation Date:
 - Last Modified Date:
 - COIN Course:
 - Grade Received:
 - Comments regarding design:

Include an introductory page that tells the COIN class each was created for, the grade received, and any comments regarding how to best run the program (i.e. the language it was written in and how it was compiled). The four CD/DVDs should be included in a 4-pack CD organizer sheet after the Title and Introductory Pages. The code shall also be presented in hard copy, well commented, with screen shots of the application at work.

- Three ethics papers from previous COIN classes. These should be printed and included after the programs section. Include an introductory page previous to the

papers that tells the COIN class for which each was created for and the grade received.

- Two presentations from COIN classes. These should be printed in either handout or full slide form and included after the ethics papers. Include an introductory page previous to the presentations that tells the COIN class that it was presented in and, if applicable, the grade received for that presentation.
- Each section shall be separated by a file divider.
- Overall Organization of Portfolio: Title Page, Program Introductory Page, CD/DVD organizer sheet with 4 CD/DVDs, Hard Copy of code with screen shots, Ethics Paper Introductory Page, 3 ethics papers, Presentation Introductory Page, two presentations.

Text: N/A

Attendance: This class meets initially with the entire Senior Project class and attendance policies are then scheduled between the student and his/her advisor.

Academic Integrity and the Honor Code: All students are expected to adhere to Charleston Southern University's code of Academic Honesty. For example, all work must be original and must not be copied from another student. Failure to follow this guideline will be considered a violation of the code and will be treated accordingly.

Grading: Portfolio – 100%

Grading Scale: Pass/Fail

Tentative Course Schedule: **Final** portfolio is due to student advisor no later than 2 weeks prior to the last day of classes. This portfolio is then reviewed among the department and a decision regarding Pass/Fail is reached by Chair no later than the first reading day. Draft portfolio reviews should be coordinated between student and advisor prior to the final submission.

Course Evaluations: CSU has implemented a new procedure of collecting course evaluations via the web. Instructions on how to access this system and how to evaluate the course will be available midway through the class. I encourage you to take this seriously and provide constructive feedback for improving the class.



Code and Programs:

CD 1-4.

All CD's contain all projects, simply because there is plenty of space, and for archival purposes.

COINS 315: Data Structures.

Instructor: John Briner, Ph.D

Grade: 80

8-puzzle.

COINS 325: Java Programming

Instructor: James Roberts

Grade: Sat.

Payroll System

java PayrollTest will run the payroll system test.

COINS 415: Advanced Algorithms

Instructor: Yu-Ju Lin, Ph.D

Grade: 75

Menu System and Line Editor

The line editor is run simply using EDIT, then the filename to edit.

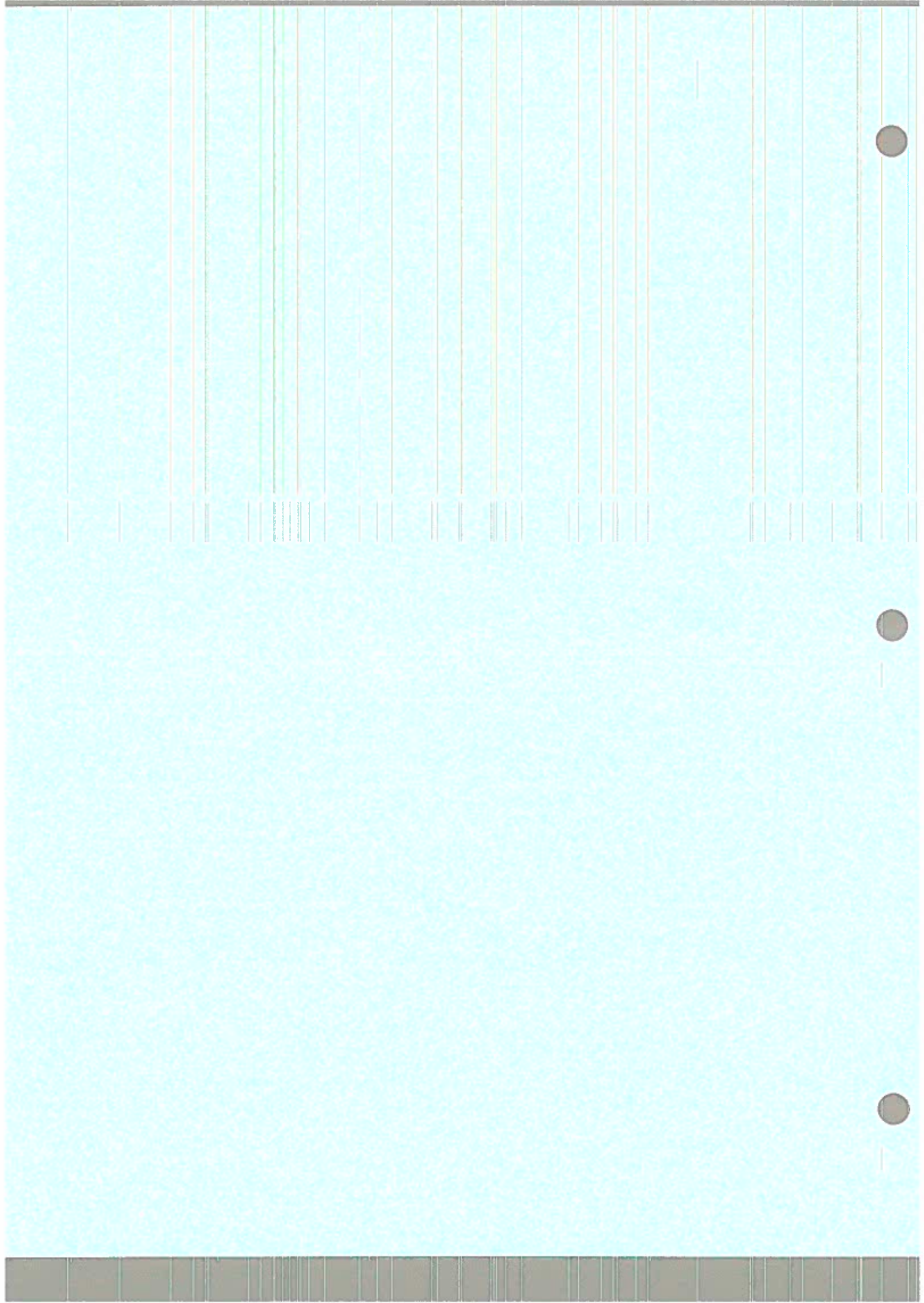
COINS 326 Java Programming II

Instructor: Valerie Sessions, PhD.

Grade: 90

Digital Flip-clock for Final Project.

This is a Java servlet. Must be run local in the browser, or special permissions set up for the graphics. Thanks to Shane for figuring that out for me.



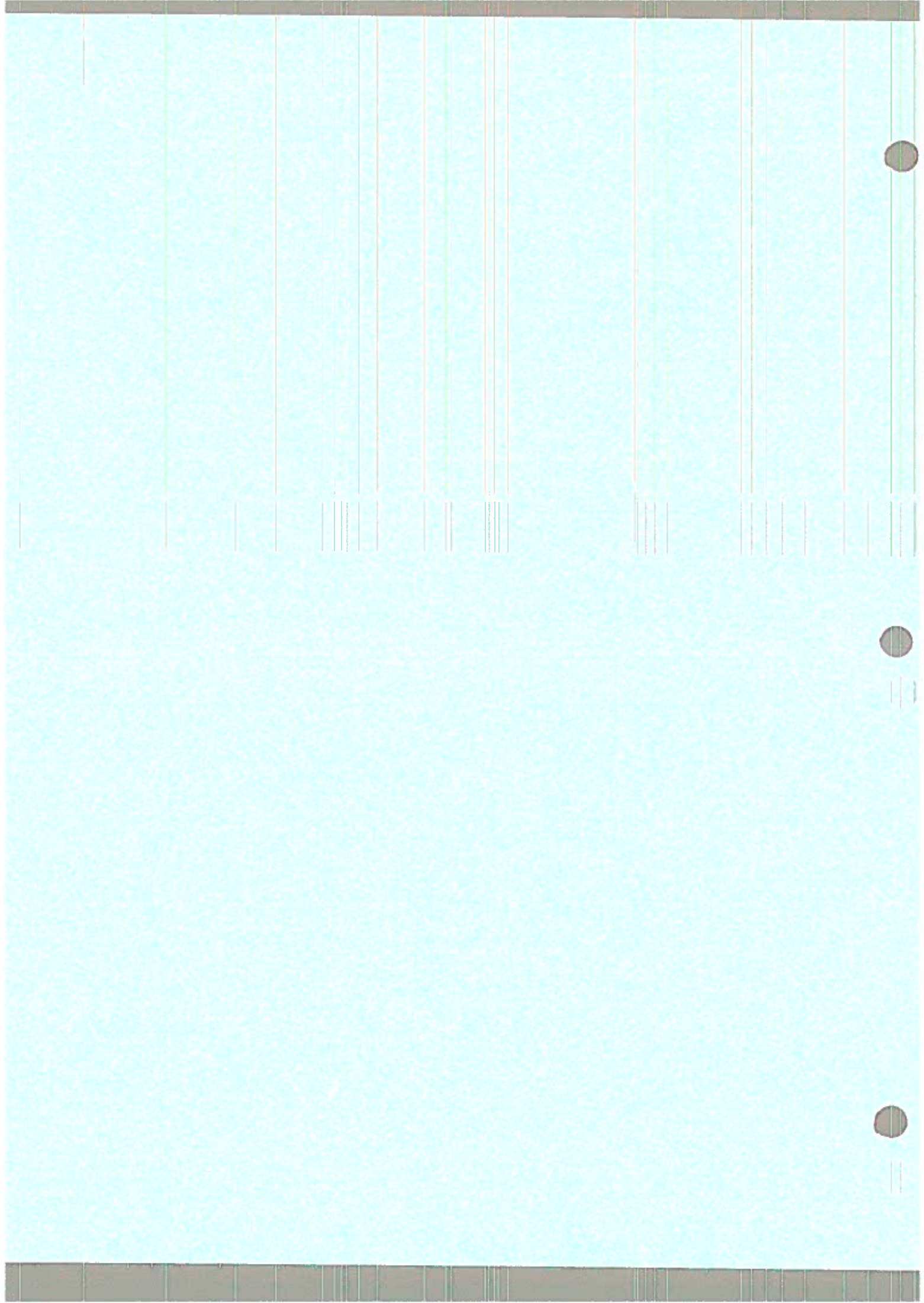

```

1  ///////////////////////////////////////////////////////////////////
2  // filename: cpuzzle.h
3  // author: smg
4  // date:[2006-03-30T19:25:17-0500]
5  // class: COINS 315
6  // instructor: Dr. Briner
7  // description: cpuzzle. actually this is the  $n^2 - 1$  puzzle
8  // this is the header file of the class.
9  // the idea: this class has three separate ways of solving an eight
10
11  ///////////////////////////////////////////////////////////////////
12  #ifndef CPUZZLE_H
13  #define CPUZZLE_H
14
15  • static int const max_states = 500;
16  • static int const MAX = 3; // this is a 3x3 puzzle.
17  • typedef int intarray[];
18
19  class cpuzzle
20  {
21  public:
22      • cpuzzle();           // constructor
23      • ~cpuzzle();         // destructor
24
25      • void cpuzzle::Do(void);
26
27      • bool Done;
28
29  private:
30      • int i,j; // init counters
31      • int l[2];
32      // the attributes
33
34      • int goal[MAX][MAX]; //goal state
35      • int start[MAX][MAX]; //start states, that will be determined randomly
36      • int expanded_states[4][MAX][MAX];
37      • int temporary[MAX][MAX]; //for temporary use
38      • int d,mvs, k,m,n;
39      • int current[max_states][MAX][MAX];
40      • int next[max_states][MAX][MAX];
41      • int t[MAX][MAX];
42      • char a;
43
44      • void t_1(int k, int a[4][MAX][MAX], int b[MAX][MAX]);
45      • void t_2(int k, int a[max_states][MAX][MAX], int b[MAX][MAX]);
46      • void i_t_1(int b[MAX][MAX], int k, int a[4][MAX][MAX]);
47      • void i_t_2(int b[MAX][MAX], int k, int a[max_states][MAX][MAX]);
48      • void display_tiles(int a[MAX][MAX]);
49      • int tiles_out(int a[MAX][MAX], int b[MAX][MAX]);
50      • int[] * find_tile(int a[MAX][MAX], int b[MAX][MAX]);
51      • int manDistance(int a[MAX][MAX], int b[MAX][MAX]);
52      • void press_a_key(void);
53      • int cust_rand(void);
54      • int pick_number(void);
55      • void copy_series(int a[MAX][MAX],int b[MAX][MAX]);

```

```
56     • void random_create(int a[MAX][MAX],int b[MAX][MAX]);
57     • void expandState(int a[MAX][MAX],int b[4][MAX][MAX]);
58     • void expandAll(int a[max_states][MAX][MAX],int b[max_states][MAX][MAX]);
59     • void clear_layer(int a[max_states][MAX][MAX]);
60     • int comp_states(int a[MAX][MAX],int b[MAX][MAX]);
61     • int cpuzzle::compare_all(int a[max_states][MAX][MAX],int b[MAX][MAX]);
62
63     public:
64
65 ]; // end class cPuzzle
66
67
68 #endif
69
```

```
1 #include <cstdlib>
2 #include <iostream>
3 #include "cpuzzle.h"
4     using namespace std;
5     cpuzzle opz;
6     int main(int argc, char *argv[])
7     {
8         opz.Do();
9         system("PAUSE");
10    ← return EXIT_SUCCESS;
11    }
12
```


```

1
2
3 public class Date {
4     • private int month; // 1-12
5     • private int day;    // 1-31 based on month
6     • private int year;  // any year
7
8     // constructor: call checkMonth to confirm proper value for month;
9     // call checkDay to confirm proper value for day
10
11     public Date( int theMonth, int theDay, int theYear ) {
12         month = checkMonth( theMonth ); // validate month
13         year = theYear; // could validate year
14         day = checkDay( theDay ); // validate day
15         System.out.printf(
16             "Date object constructor for date %s\n", toString() )
17     } // end Date constructor
18
19     // utility method to confirm proper month value
20     private int checkMonth( int testMonth ) {
21         if ( testMonth > 0 && testMonth <= 12 ) // validate month
22             return testMonth;
23         else // month is invalid
24         {
25             System.out.printf( "Invalid month (%d) set to 1.\n", testMonth );
26             return 1; // maintain object in consistent state
27         } // end else
28     } // end method checkMonth
29
30     // utility method to confirm proper day value based on month and year
31     private int checkDay( int testDay ) {
32         int daysPerMonth[] =
33         {
34             0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
35
36         // check if day in range for month
37         if ( testDay > 0 && testDay <= daysPerMonth[ month ] )
38             return testDay;
39
40         // check for leap year
41         if ( month == 2 && testDay == 29 && ( year % 400 == 0 ||
42             ( year % 4 == 0 && year % 100 != 0 ) ) )
43             return testDay;
44
45         System.out.printf( "Invalid day (%d) set to 1.\n", testDay );
46
47         return 1; // maintain object in consistent state
48     } // end method checkDay
49

```

```
50 // return day
51 public int getDay() {
52     return day;
53 } // end method getDay
54
55 // return month
56 public int getMonth() {
57     return month;
58 } // end method getMonth
59
60 // return year
61 public int getYear() {
62     return year;
63 } // end method getYear
64
65 // return a String of the form month/day/year
66 public String toString() {
67     return String.format( "%d/%d/%d", month, day, year );
68 } // end method toString
69 } // end class Date
70
71
```

```

1  // filename: BasePlusCommissionEmployee.java
2  // author: sgarbeil
3  // purpose: lab Challenge1
4  // description : calculates a commission for any employee
5  // whether salary or base.
6
7  public class BasePlusCommissionEmployee extends CommissionEmployee {
8      • private double baseSalary; // base salary per week
9
10     // nine-argument constructor
11     public BasePlusCommissionEmployee( String first, String last,
12                                       String ssn, int month, int day, int year,
13                                       double sales, double rate, double salary )
14     {
15         super( first, last, ssn, month, day, year, sales, rate );
16         setBaseSalary( salary );
17     } // end nine-argument BasePlusCommissionEmployee constructor
18
19     // set base salary
20     public void setBaseSalary( double salary )
21     {
22         baseSalary = salary < 0.0 ? 0.0 : salary; // non-negative
23     } // end method setBaseSalary
24
25     // return base salary
26     public double getBaseSalary()
27     {
28         return baseSalary;
29     } // end method getBaseSalary
30
31     // calculate earnings; override method earnings in CommissionEmployee
32     public double earnings()
33     {
34         return getBaseSalary() + super.earnings();
35     } // end method earnings
36
37     // return String representation of BasePlusCommissionEmployee object
38     public String toString()
39     {
40         return String.format( "%s %s; %s: $%,.2f",
41                               "base-salaried", super.toString(),
42                               "base salary", getBaseSalary() );
43     } // end method toString
44 } // end class BasePlusCommissionEmployee
45

```



```

1  // filename: CommissionEmployee.java
2  // author: sgarbeil
3  // purpose: lab Challenge1
4  // description : calculates a commission for any employee
5  // whether salary or base.
6
7  public class CommissionEmployee extends Employee {
8      • private double grossSales; // gross weekly sales
9      • private double commissionRate; // commission percentage
10
11     // eight-argument constructor
12     public CommissionEmployee( String first, String last, String ssn,
13                               int month, int day, int year, double sales, double rate
14     {
15         — super( first, last, ssn, month, day, year );
16         — setGrossSales( sales );
17         — setCommissionRate( rate );
18     } // end eight-argument CommissionEmployee constructor
19
20     // set commission rate
21     public void setCommissionRate( double rate )    {
22         — commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
23     } // end method setCommissionRate
24
25     // return commission rate
26     public double getCommissionRate()    {
27         — return commissionRate;
28     } // end method getCommissionRate
29
30     // set gross sales amount
31     public void setGrossSales( double sales )
32     {
33         — grossSales = sales < 0.0 ? 0.0 : sales;
34     } // end method setGrossSales
35
36     // return gross sales amount
37     public double getGrossSales()
38     {
39         — return grossSales;
40     } // end method getGrossSales
41
42     // calculate earnings; override abstract method earnings in Employee
43     public double earnings()
44     {

```

```
44  ← return getCommissionRate() * getGrossSales();
45  } // end method earnings
46
47  // return String representation of CommissionEmployee object
48  public String toString()
49  {
50  ← return String.format( "%s: %s\n%s: $%,.2f; %s: %.2f",
51                          "commission employee", super.toString(),
52                          "gross sales", getGrossSales(),
53                          "commission rate", getCommissionRate() );
54  } // end method toString
55  } // end class CommissionEmployee
56
57
```



```
1 // filename: CommissionEmployee.java
2 // author: sgarbeil
3 // purpose: lab 10
4 // description : calculates a commission for any employee
5 // whether salary or base.
6 // Date class declaration with get methods added.
7 public abstract class Employee {
8     • private String firstName;
9     • private String lastName;
10    • private String socialSecurityNumber;
11    • private Date birthDate;
12
13    // six-argument constructor
14    public Employee( String first, String last, String ssn,
15                  int month, int day, int year ) {
16        — firstName = first;
17        — lastName = last;
18        — socialSecurityNumber = ssn;
19        — birthDate = new Date( month, day, year );
20    } // end six-argument Employee constructor
21
22    // set first name
23    public void setFirstName( String first ) {
24        — firstName = first;
25    } // end method setFirstName
26
27    // return first name
28    public String getFirstName() {
29        ← return firstName;
30    } // end method getFirstName
31
32    // set last name
33    public void setLastName( String last ) {
34        — lastName = last;
35    } // end method setLastName
36
37    // return last name
38    public String getLastName() {
39        ← return lastName;
40    } // end method getLastName
41
42    // set social security number
43    public void setSocialSecurityNumber( String ssn ) {
```



```
44     socialSecurityNumber = ssn; // should validate
45 } // end method setSocialSecurityNumber
46
47 // return social security number
48 public String getSocialSecurityNumber() {
49     return socialSecurityNumber;
50 } // end method getSocialSecurityNumber
51
52 // set birth date
53 public void setBirthDate( int month, int day, int year ) {
54     birthDate = new Date( month, day, year );
55 } // end method setBirthDate
56
57 // return birth date
58 public Date getBirthDate() {
59     return birthDate;
60 } // end method getBirthDate
61
62 // return String representation of Employee object
63 public String toString() {
64     return String.format( "%s %s\n%s: %s\n%s: %s",
65                           getFirstName(), getLastName(),
66                           "social security number", getSocialSecurityNumber(
67                           "birth date", getBirthDate() );
68     } // end method toString
69
70 // abstract method overridden by subclasses
71 public abstract double earnings();
72 } // end abstract class Employee
73
```

```

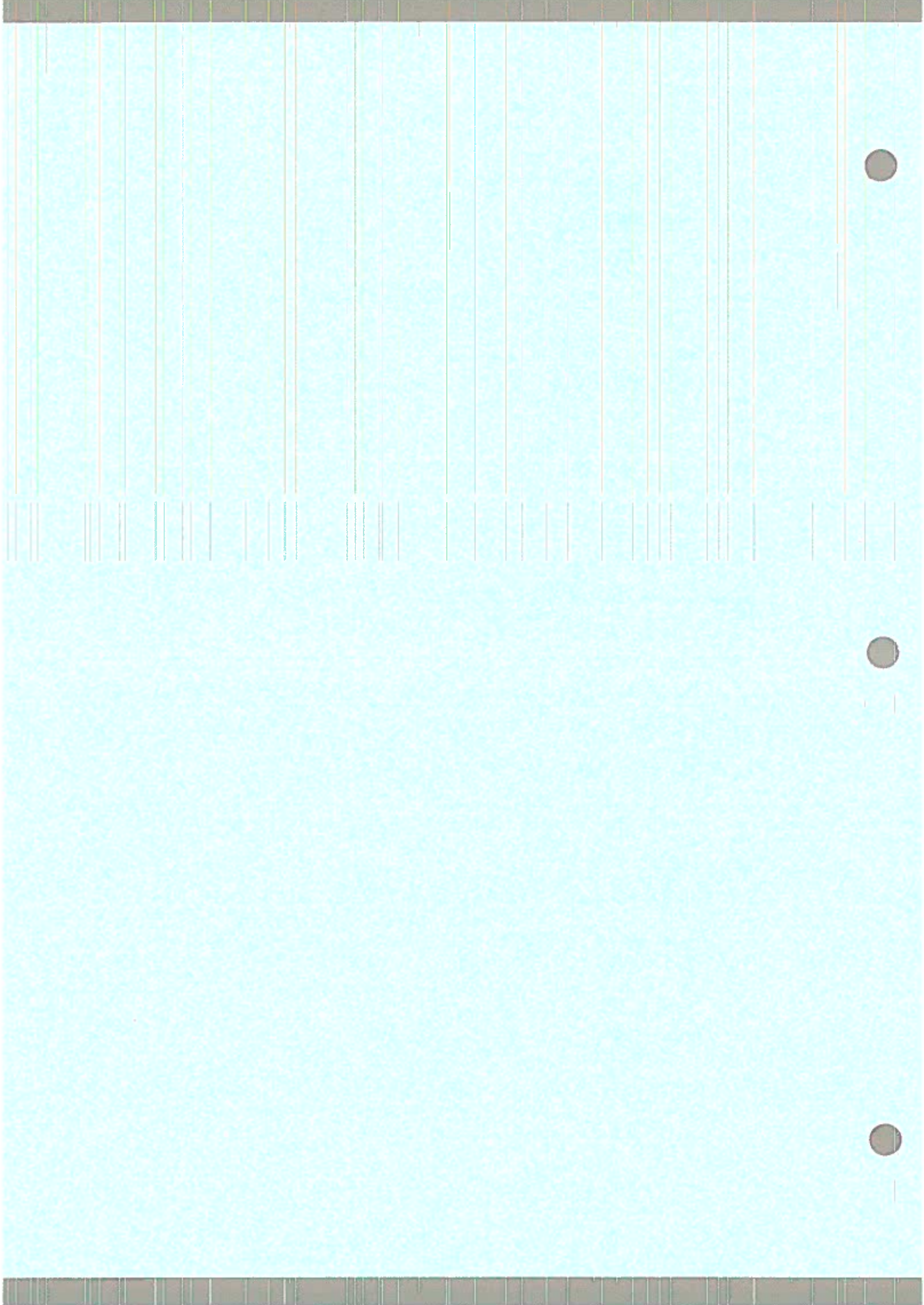
1  // filename: HourlyEmployee.java;
2  // author: sgarbeil w/help from Cary Humphries
3  // date: 12/1/05
4  // purpose: to demonstrate the ability to create a reference object
5  // array and to limit the utility of cut and paste comments
6
7  // description: This is a test unit
8  // status: appears to work.
9
10 public class HourlyEmployee extends Employee
11 {
12     • private double wage; // wage per hour
13     • private double hours; // hours worked for week
14
15     // eight-argument constructor
16     public HourlyEmployee( String first, String last, String ssn,
17                           int month, int day, int year,
18                           double hourlyWage, double hoursWorked )
19     {
20         — super( first, last, ssn, month, day, year );
21         — setWage( hourlyWage );
22         — setHours( hoursWorked );
23     } // end eight-argument HourlyEmployee constructor
24
25     // set wage
26     public void setWage( double hourlyWage )
27     {
28         — wage = hourlyWage < 0.0 ? 0.0 : hourlyWage;
29     } // end method setWage
30
31     // return wage
32     public double getWage()
33     {
34     ← return wage;
35     } // end method getWage
36
37     // set hours worked
38     public void setHours( double hoursWorked )
39     {
40         — hours = ( ( hoursWorked >= 0.0 ) && ( hoursWorked <= 168.0 ) ) ?
41             hoursWorked : 0.0;
42     } // end method setHours
43
44     // return hours worked
45     public double getHours()

```



```
46  {  
47  ← return hours;  
48  } // end method getHours  
49  
50  // calculate earnings; override abstract method earnings in Employee  
51  public double earnings()  
52  {  
53  ← if ( getHours() <= 40 ) // no overtime  
54  ← return getWage() * getHours();  
55  ← else  
56  ← return 40 * getWage() + ( getHours() - 40 ) * getWage() * 1.5;  
57  } // end method earnings  
58  
59  // return String representation of HourlyEmployee object  
60  public String toString()  
61  {  
62  ← return String.format( "hourly employee: %s\n%s: $%,.2f; %s: $%,.2f",  
63  ← super.toString(), "hourly wage", getWage(),  
64  ← "hours worked", getHours() );  
65  } // end method toString  
66  } // end class HourlyEmployee  
67  
68
```

```
1
2
3 public class SalariedEmployee extends Employee {
4     • private double weeklySalary;
5
6     // seven-argument constructor
7     public SalariedEmployee( String first, String last, String ssn,
8                             int month, int day, int year, double salary ) {
9         super( first, last, ssn, month, day, year );
10        setWeeklySalary( salary );
11    } // end seven-argument SalariedEmployee constructor
12
13    // set salary
14    public void setWeeklySalary( double salary ) {
15        weeklySalary = salary < 0.0 ? 0.0 : salary;
16    } // end method setWeeklySalary
17
18    // return salary
19    public double getWeeklySalary() {
20        return weeklySalary;
21    } // end method getWeeklySalary
22
23    // calculate earnings; override abstract method earnings in Employee
24    public double earnings() {
25        return getWeeklySalary();
26    } // end method earnings
27
28    // return String representation of SalariedEmployee object
29    public String toString() {
30        return String.format( "salaried employee: %s\n%s: $%,.2f",
31                              super.toString(), "weekly salary", getWeeklySalary() );
32    } // end method toString
33 } // end class SalariedEmployee
34
35
```

PayrollSystemTest.java

```
// filename: PayrollSystemTest.java
// author: sgarbeil
// purpose: lab Challenge1
// description : Tests the payroll systeme
// Employee hierarchy test program.

import java.util.Scanner; // program uses Scanner to obtain user input
import java.io.*;

public class PayrollSystemTest {
    public static void main( String args[] )    {
        // create subclass objects
        SalariedEmployee salariedEmployee =
            new SalariedEmployee(
                "John", "Smith", "111-11-1111", 6, 15, 1944, 800.00 );
        HourlyEmployee hourlyEmployee =
            new HourlyEmployee(
                "Karen", "Price", "222-22-2222", 12, 29, 1960, 16.75, 40 );
        CommissionEmployee commissionEmployee =
            new CommissionEmployee(
                "Sue", "Jones", "333-33-3333", 9, 8, 1954, 10000, .06 );
        BasePlusCommissionEmployee basePlusCommissionEmployee =
            new BasePlusCommissionEmployee(
                "Bob", "Lewis", "444-44-4444", 3, 2, 1965, 5000, .04, 300 );

        System.out.println( "Employees processed individually:\n" );

        System.out.printf( "%s\n%s: $%,.2f\n\n",
            salariedEmployee, "earned", salariedEmployee.earnings() );
        System.out.printf( "%s\n%s: $%,.2f\n\n",
            hourlyEmployee, "earned", hourlyEmployee.earnings() );
        System.out.printf( "%s\n%s: $%,.2f\n\n",
            commissionEmployee, "earned", commissionEmployee.earnings() );
        System.out.printf( "%s\n%s: $%,.2f\n\n",
            basePlusCommissionEmployee,
            "earned", basePlusCommissionEmployee.earnings() );

        // create four-element Employee array
        Employee employees[] = new Employee[ 4 ];

        // initialize array with Employees
        employees[ 0 ] = salariedEmployee;
        employees[ 1 ] = hourlyEmployee;
        employees[ 2 ] = commissionEmployee;
        employees[ 3 ] = basePlusCommissionEmployee;

        Scanner input = new Scanner( System.in ); // to get current month
        int currentMonth;

        // get and validate current month
        do {
            System.out.print( "Enter the current month (1 - 12): " );
            currentMonth = input.nextInt();
            System.out.println();
        } while ( ( currentMonth < 1 ) || ( currentMonth > 12 ) );

        System.out.println( "Employees processed polymorphically:\n" );

        // generically process each element in array employees
        for ( Employee currentEmployee : employees ) {
            System.out.println( currentEmployee ); // invokes toString

            // determine whether element is a BasePlusCommissionEmployee

```



```

PayrollSystemTest.java
if ( currentEmployee instanceof BasePlusCommissionEmployee )
    // downcast Employee reference to
    // BasePlusCommissionEmployee reference
    BasePlusCommissionEmployee employee =
        ( BasePlusCommissionEmployee ) currentEmployee;

    double oldBaseSalary = employee.getBaseSalary();
    employee.setBaseSalary( 1.10 * oldBaseSalary );
    System.out.printf(
        "new base salary with 10%% increase is: $%,.2f\n",
        employee.getBaseSalary() );
} // end if

// if month of employee's birthday, add $100 to salary
if ( currentMonth == currentEmployee.getBirthDate().getMonth() ) {
    System.out.printf(
        "earned $%,.2f %s\n\n", currentEmployee.earnings(),
        "plus $100.00 birthday bonus" );
}
else {
    System.out.printf(
        "earned $%,.2f\n\n", currentEmployee.earnings() );
} // end else
} // end for

// get type name of each object in employees array
for ( int j = 0; j < employees.length; j++ ) {
    System.out.printf( "Employee %d is a %s\n", j,
        employees[ j ].getClass().getName() );
} // end for
} // end main
} // end class PayrollSystemTest

```


File: flipclock.java

```

/*
 * @(#)flipclock.java
 * @flipclock class.
 * @course: COINS 326 Java and Multimedia
 * @instructor: Valerie Sessions, Ph.D
 *
 *
 * @author Steven Garbeil, Peasant
 * @version 1.01 2007/12/3
 * @comments
 * Flipclock: Do you remember the old seventies clocks with the
 * panels? Well this is supposed to be one. I had this all mapped
 * out and working... but I waited too long and didn't comment. I
 * sure should know better The general idea is that it displays
 * the time. Actually it's simpler than that. It actually grabs the time off
 * the system clock, slices up the digits and determines the values.
 * at random intervals, it flips the leaves
 * @todo next trick is going to be seconds on a moving tape
 * @todo alarm
 */
import java.applet.*;
import java.util.*;
import java.awt.*;
import java.awt.Dimension;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.Graphics;
import java.util.Calendar;
import java.awt.Image;
import javax.swing.JApplet;
import javax.swing.ImageIcon;
import javax.swing.JSlider;
import javax.swing.Timer;

public class flipclock extends JApplet implements Runnable {

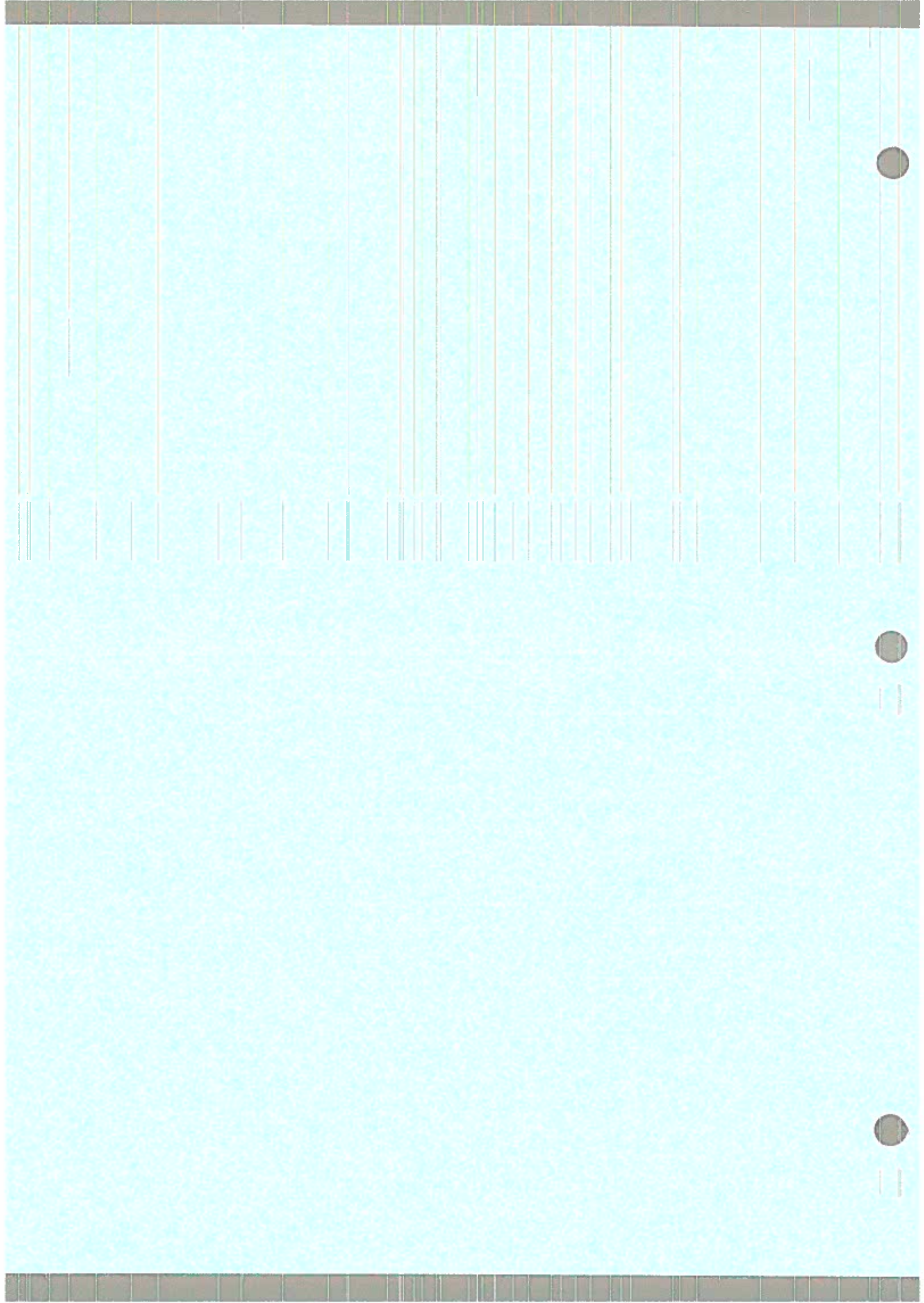
    • public Image offscreen;

    // set of constants that identify the digits

    • final int  HOURSTENS = 0;
    • final int  HOURSONES = 1;
    • final int  MINUTETENS = 2;
    • final int  MINUTESONES = 3;
    • final int  SECONDSTENS = 4;
    • final int  SECONDSONES = 5;

    • public Thread  clock_thread = null;
    • public int     sleep_time = 100;           // update 30( or so ) times a second.
    • public Date    date;                       // create a date variable
    • public boolean show_seconds_p = false;
    • public boolean twenty_four_hour_p = false;

```



File: flipclock.java

```

- public int      time_zone_offset;
- public int      time_zone_offset_milliseconds;
- public Image    clockShell;
- public Image    numbers[] = new Image[40];      // numeros
- public Image    flips[] = new Image[20];        // the flips
- public Image    clockdigits[] = new Image[6];    // the display
- public Image    flipsignal;
- public boolean  Done = false;
- public int      width;
- public int      height;
// date ... that is Date methods don't GET the time they return the time stored in the
// date class. so to keep the clock a running you MUST call date.setTime(whatever).
// these variables are for dividing the time up into manageable slices.
- public int      hours;// = date.getHours();      // current hours
- public int      minutes;// = date.getMinutes();   // current minutes
- public int      seconds;// = date.getSeconds();   // current seconds
- public int      hourstens;// = (hours / 10);
- public int      hoursones;// = (hours % 10);
- public int      minstens;// = (minutes / 10);
- public int      minsones;// = (minutes % 10);
- public int      secondtens;// = (seconds /10);
- public int      secondones;// = (seconds % 10);
- public int      flipsecondtens;// = (seconds /10);
- public int      flipsecondones;// = (seconds % 10);
- public Boolean  flipping[] = new Boolean[6];      // flipping array for each digit
- public Boolean  FlipNow = false;
- public int      scalex;
- public int      scaley;

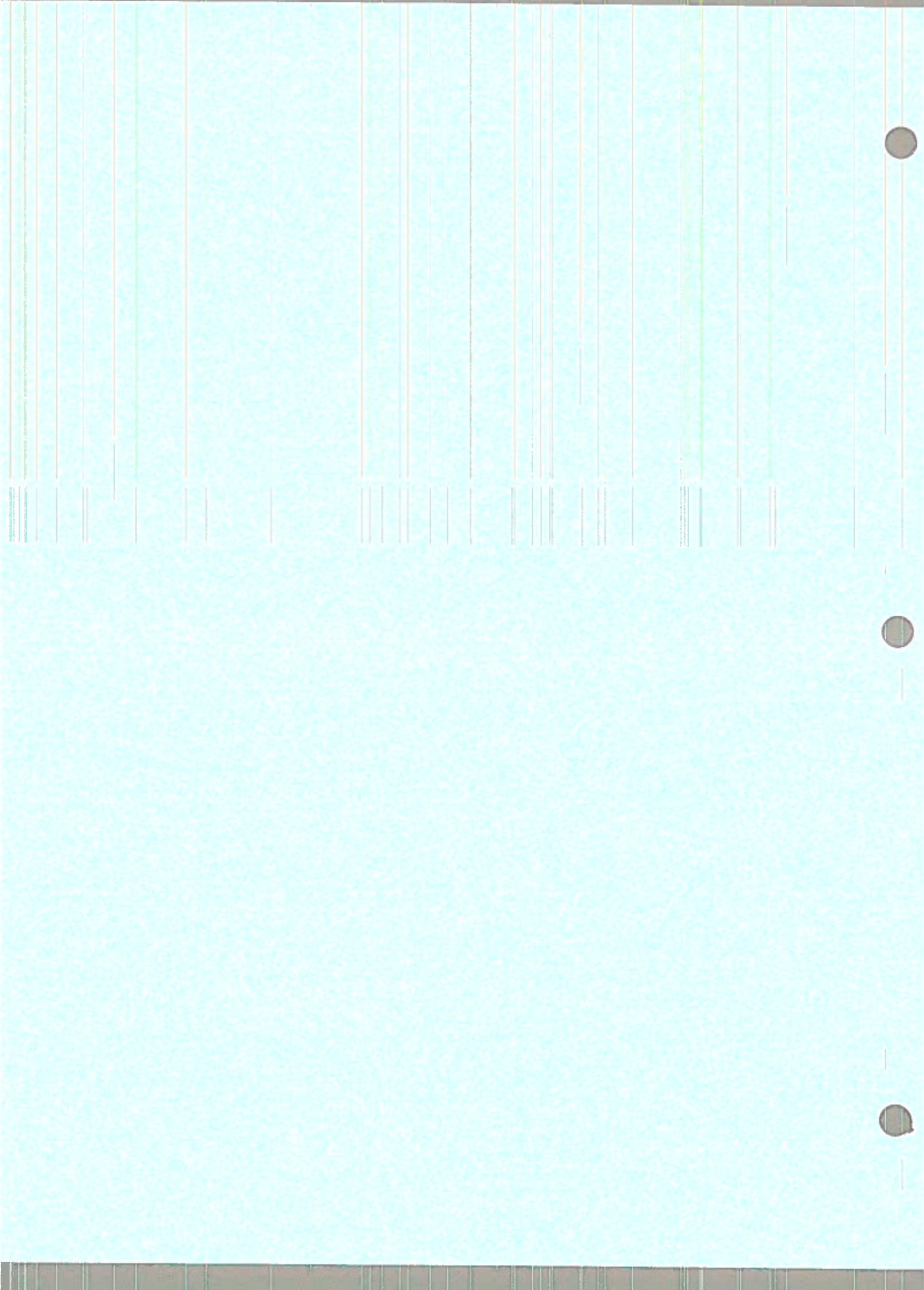
- AudioClip whir;
- AudioClip flip;
- AudioClip bong;
- AudioClip kling;
- AudioClip okay;

/** Initialization method that will be called after the applet is loaded
 * into the browser.
 */

- public flipclock() {
-     date = new Date();          // sets the date
- }

- public void init() {
-     // TODO start asynchronous download of heavy resources
-     super.init();
-     whir = getAudioClip(getDocumentBase(), ".\\sounds\\whir.wav");
-     flip = getAudioClip(getDocumentBase(), ".\\sounds\\clunk.wav");
-     bong = getAudioClip(getDocumentBase(), ".\\sounds\\bong.wav");
-     kling = getAudioClip(getDocumentBase(), ".\\sounds\\kling.wav");
-     okay = getAudioClip(getDocumentBase(), ".\\sounds\\ok.wav");
-     flipsignal = new ImageIcon(".\\graphics\\flipsignal.gif").getImage();
-     clockShell = new ImageIcon( ".\\graphics\\flipclock.jpg" ).getImage();

```

File: flipclock.java

```

    for (int i = 0; i < 15; i++) {           // load a bunch of images
        numbers[i] = new ImageIcon(".\\graphics\\" + "glyph" + Integer.toString(i)
                                   + ".gif").getImage();
    } // end for

    for (int i = 0; i < 14; i++) {           // load the flips
        flips[i] = new ImageIcon(".\\graphics\\flip" + Integer.toString(i)
                                 + ".gif").getImage();
    } // end for

    for (int i = 0; i < 6 ; i++) {
        flipping[i]=false;                 // turn off the indicators
    }

    width = numbers[0].getWidth(null); // all images have the same height.
    height = numbers[0].getHeight(null);
    scalex = width/11;
    scaley = height/8;
} // end init

// start overrides start.

public void start(){ // runs on init.
    if (clock_thread == null) {
        clock_thread = new Thread(this);
        clock_thread.start();
    }
    okay.play();
} //

// after much reading and beating of breasts

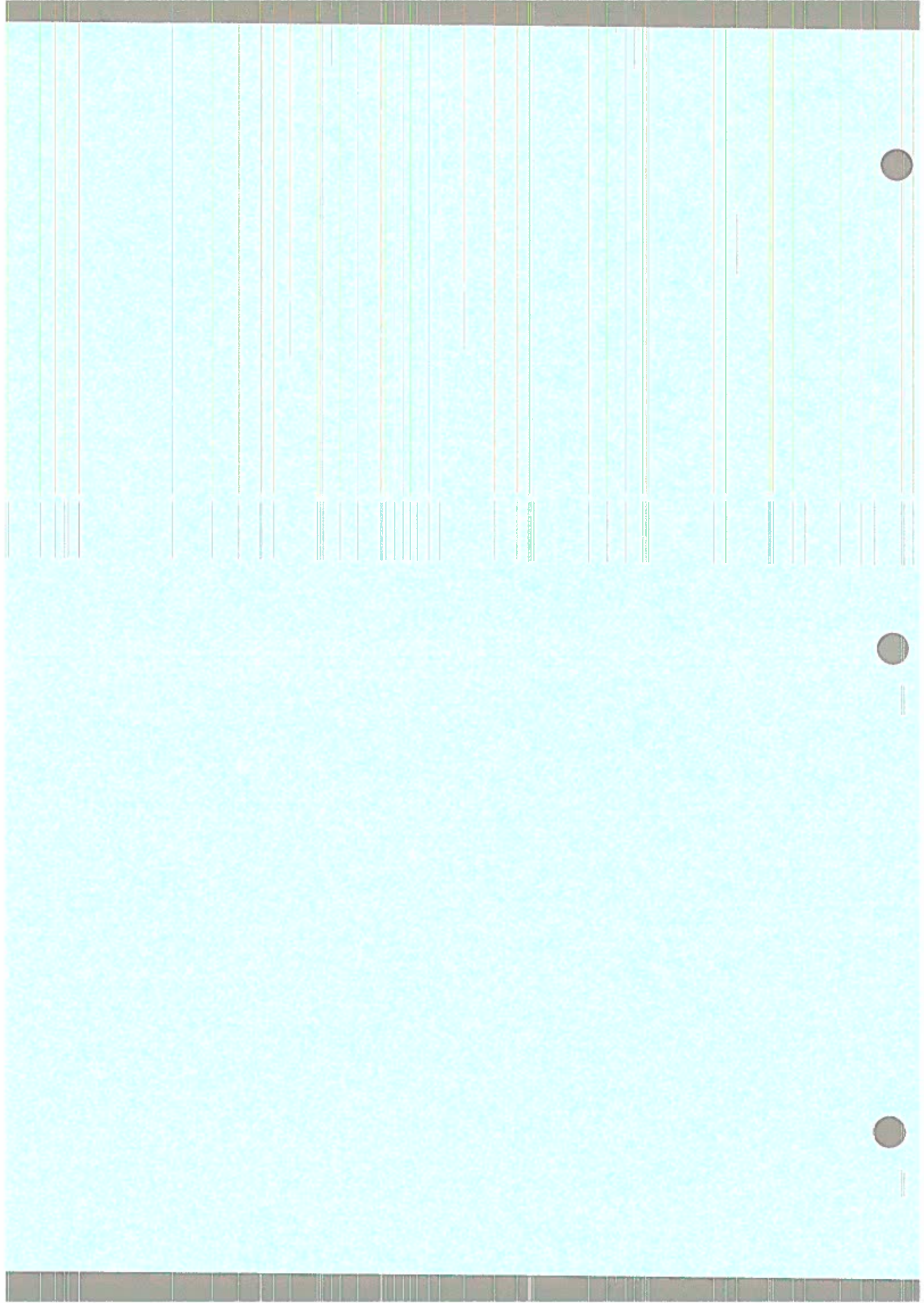
public void update(Graphics g) {           // overrides update
    updateClock(g);
}

public void run() {
    while(true) {
        try { Thread.sleep(sleep_time); }
        catch (InterruptedException e) {
            System.exit(1); }
        update(this.getGraphics());
    } // run
}

// stop gets executed when

public void stop() {
    if ((clock_thread != null) &&
        (clock_thread.isAlive()))

```



File: flipclock.java

```

    clock_thread.stop();
    clock_thread = null;
    whir.stop();
}

```

```

public void updateClock(Graphics g) {
    updateTimeRecord();
    updateDoubleBuffered(g);
}

```

```

public void updateTimeRecord() {
    date.setTime(System.currentTimeMillis() +
        time_zone_offset_milliseconds);
}

```

```

public void drawClock(Graphics g) {

    // step one: retrieve the numbers from the time
    // this is not the most efficient way of doing this, however
    // I will improve it
    hours = date.getHours();           // current hours
    minutes = date.getMinutes();       // current minutes
    seconds = date.getSeconds();       // current seconds
    hourstens = (hours / 10);
    hoursones = (hours % 10);
    mintens = (minutes / 10);
    minones = (minutes % 10);
    secondtens = (seconds / 10);
    secondones = (seconds % 10);

    // step two: draw the numbers on the browser window
    // lets get the time up

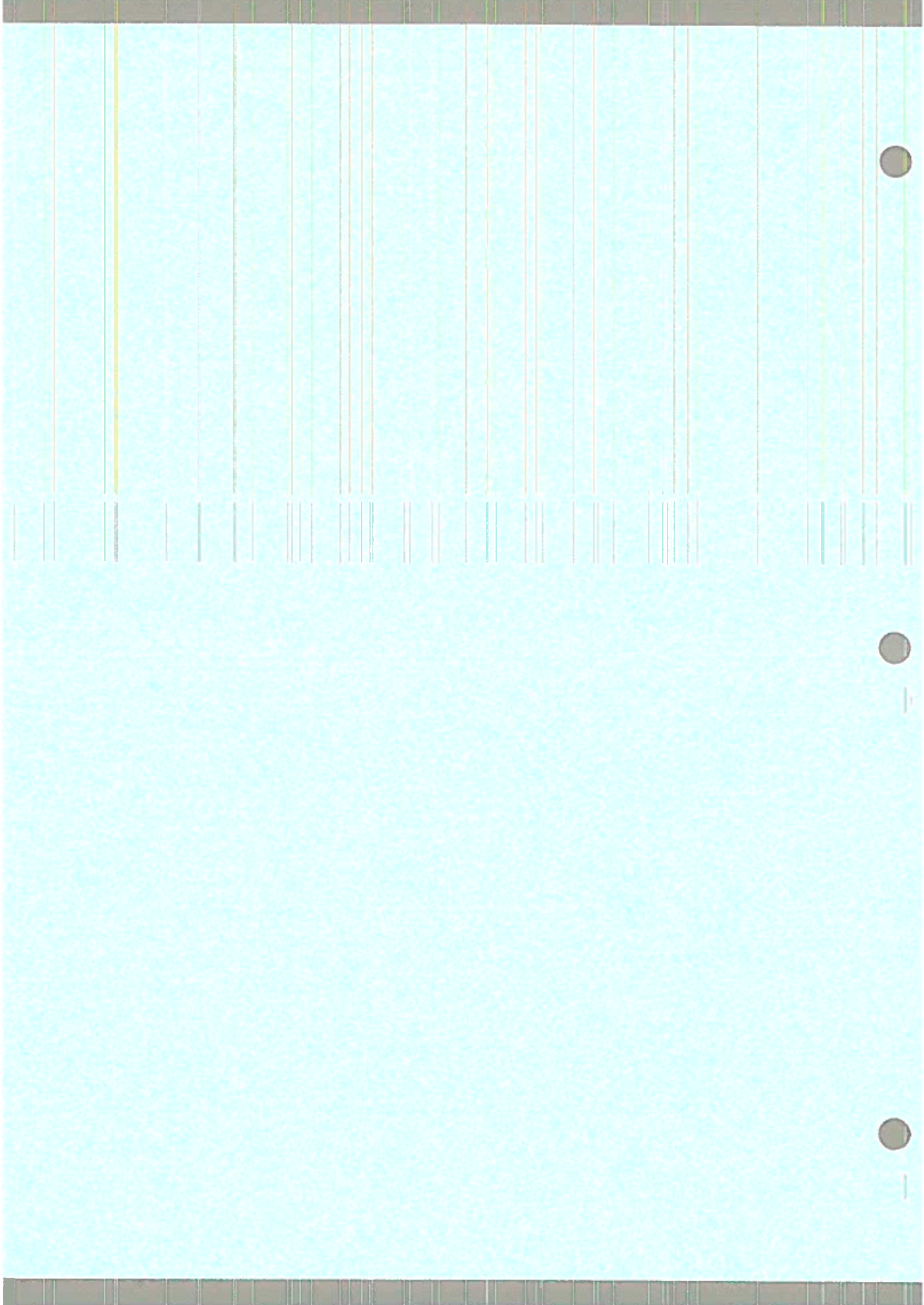
    int initx = 47;                    // this is the initial start point from the c
    int inity = 71;                    // this is the height of the y numbers
    int border1 = 36;
    int border2 = 80;
    int levely = inity;                //

    // the hours appear to be in an acceptable position
    int hourstensx = initx;            // border
    int hoursonesx = initx + scalex;

    int mintenx = border1 + initx + ( 2 * scalex ); // this is border + width of num
    int minonesx = border1 + initx + ( 3 * scalex ); // border + width of two number.
    int sectenx = border2 + initx + ( 4 * scalex ); // this is border + width of num
    int seconesx = border2 + initx + ( 5 * scalex ); // border + width of two number.

    /*
    * so we copy the image from an array depending on the clock
    * position. there are several cooler looking ways to do

```



```

* this, but this is about the simplest and easiest to
* understand, and the others (objects, arrays whatever) aren't
* any faster. I tried.
*
*
*/

/* this is a last pass kind of thing. if the flip second
* doesn't match, it flips the seconds. it was the easiest way
* to fake it.
*/
if (flipsecondones != secondones && !FlipNow) {
    FlipNow = true;
    flipping[SECONDSONES] = true;
    whir.stop();
    flip.play();

    if (secondones >= 9) {
        flipping[SECONDTENS] = true;
    }
    if (secondtens >= 5) {
        flipping[MINUTESONES] = true;
    }
    if (minsones >= 9) {
        flipping[MINUTETENS] = true;
    }
    if (minstens >= 5) {
        flipping[HOURLSONES] = true;
        kling.play();
    }

    // this is why the tens were flickering. I don't entirely
    // understand why that should be

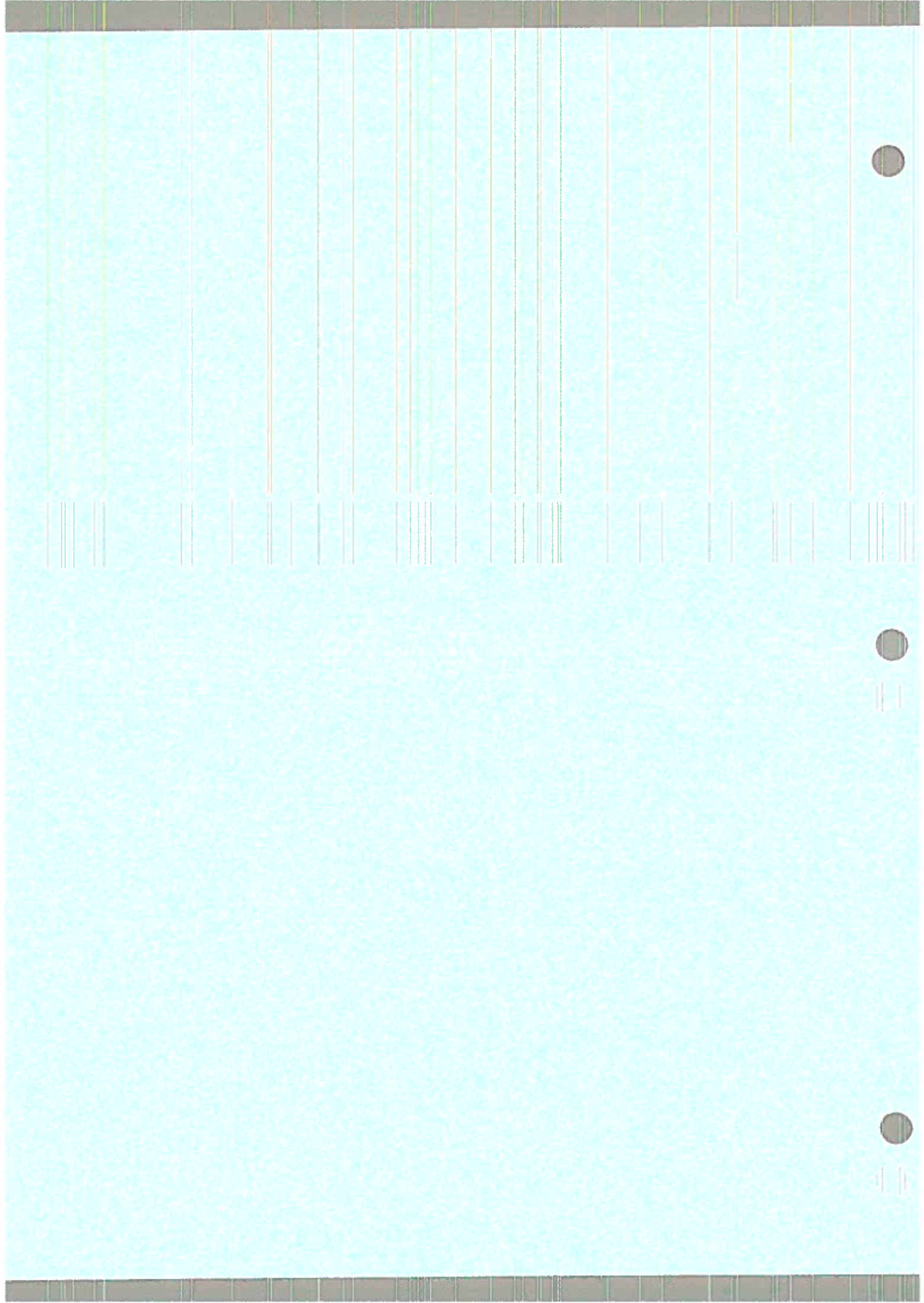
    // if (hours % 10 >= 9) {
    //     flipping[HOURLTENS] = true;
    // }
    whir.loop();
}
else if (seconds >= 2) {
    FlipNow = false; // done flipping
    for (int i = 0; i < 6; i++) {
        flipping[i] = false; // turn off the indicators
    }
}

if (flipping[HOURLTENS]) //
    clockdigits[HOURLTENS] = flips[hourstens];
else
    clockdigits[HOURLTENS] = numbers[hourstens];

if (flipping[HOURLSONES])
    clockdigits[HOURLSONES] = flips[hoursones];
else
    clockdigits[HOURLSONES] = numbers[hoursones];

if (flipping[MINUTETENS])
    clockdigits[MINUTETENS] = flips[minstens];

```

```

    } else
    {
        clockdigits[MINUTESTENS] = numbers[minstens];
    }

    if (flipping[MINUTESONES])
        clockdigits[MINUTESONES] = flips[minsones];
    else
        clockdigits[MINUTESONES] = numbers[minsones];

    if (flipping[SECONDESTENS])
        clockdigits[SECONDESTENS] = flips[secondtens];
    else
        clockdigits[SECONDESTENS] = numbers[secondtens];
    // we won't flip the ones. it's gonna be a tape anyway

    clockdigits[SECONDSONES] = numbers[secondone];

    g.drawImage(clockShell, 0, 0, this);
    g.drawImage(clockdigits[HOURLSTENS],    hourstensex,levely,scalex,scaley, this);
    g.drawImage(clockdigits[HOURLSONES],    hoursonesx,levely,scalex,scaley, this);
    g.drawImage(clockdigits[MINUTESTENS],    mintenx,levely,scalex,scaley, this);
    g.drawImage(clockdigits[MINUTESONES],    minonesx,levely,scalex,scaley, this);
    g.drawImage(clockdigits[SECONDESTENS],    sectenx,levely,scalex,scaley, this);
    g.drawImage(clockdigits[SECONDSONES],    seconesx,levely,scalex,scaley, this);
    flipsecondtens = (seconds /10);          // milliseconds aren't available so
    flipsecondone  = (seconds % 10);         // we check the last pass

} // herein ENDS the tale of drawclock()
// paint routine

public void paint(Graphics g) {
    super.paint(g);
}

// Do the drawing to an offscreen buffer -- maybe

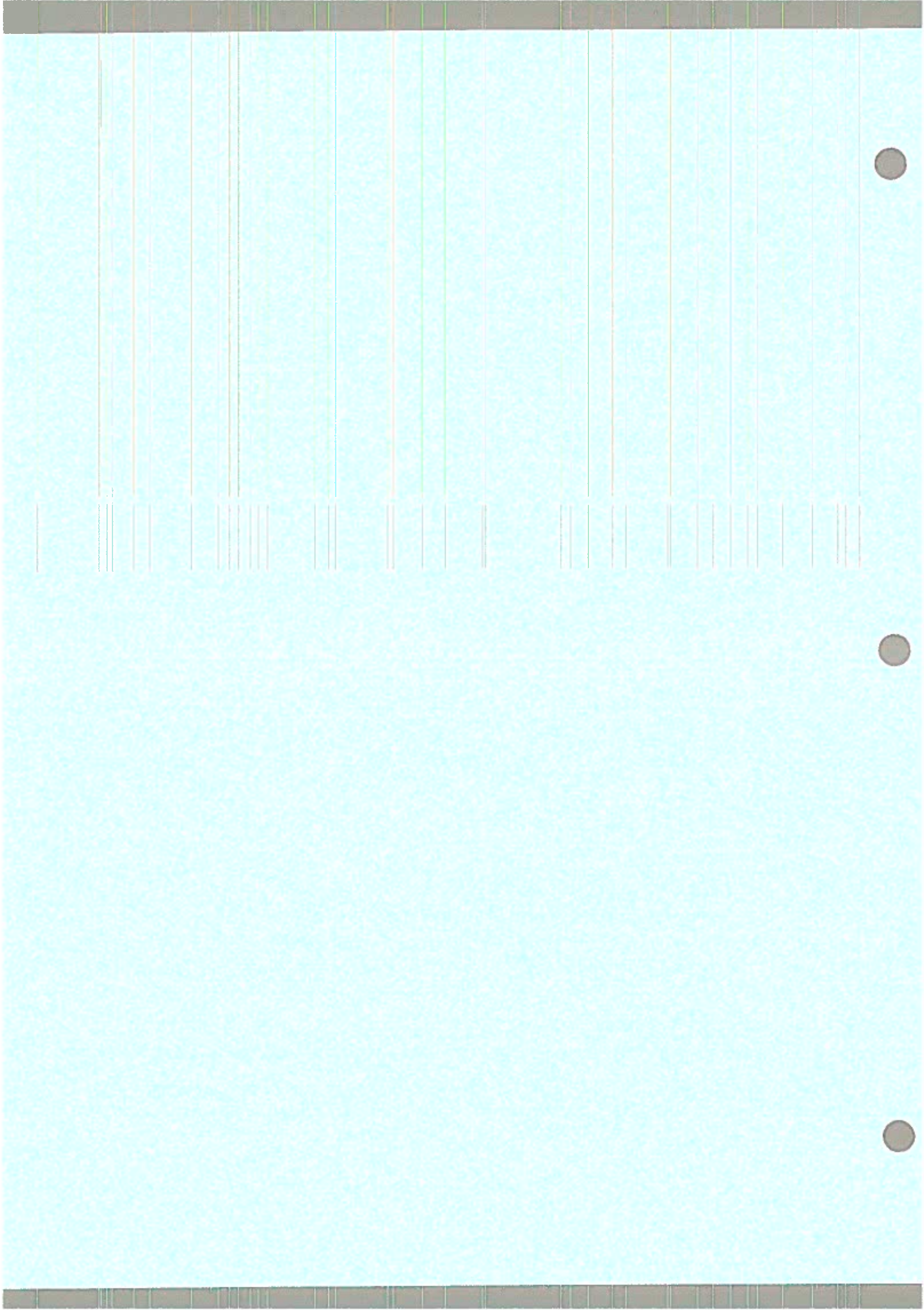
private void updateDoubleBuffered( Graphics g ) {

    // Let's make sure we have an offscreen buffer, and that
    // it's the right size.  If the applet has been resized,
    // our buffer will be the wrong size and we need to make
    // a new one

    Dimension d = getSize();
    if (offscreen == null ||
        width != d.width || height!=d.height || offscreen==null) {
        width  = d.width;
        height = d.height;
        if (width>0 || height>0) {
            offscreen = createImage( width, height );
        }
        else offscreen = null;
    }

    // If we still don't have one, give up
    if (offscreen == null)

```




```
    return;

    // Get the off-screen graphics object
    Graphics gg = offscreen.getGraphics();

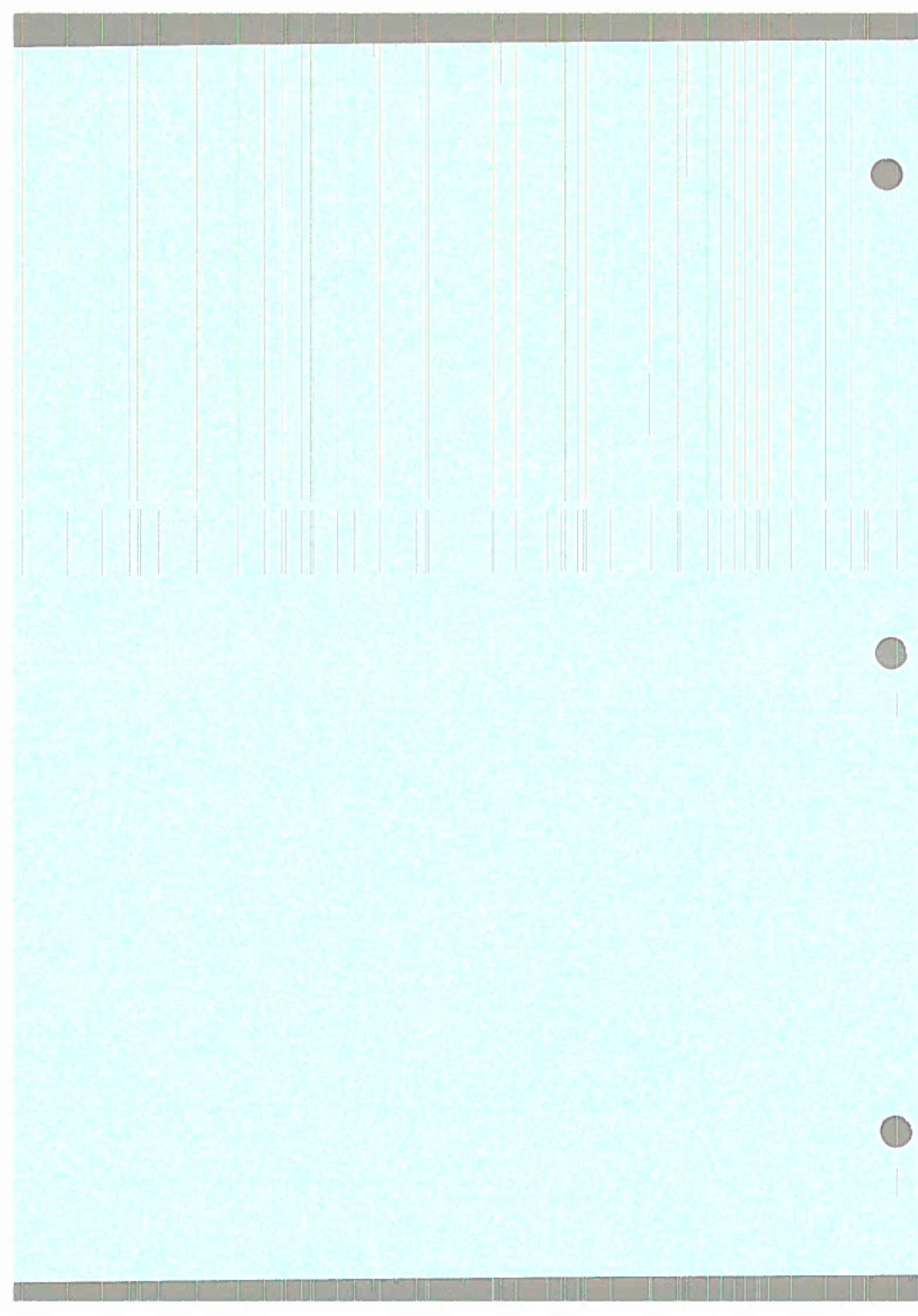
    // Clear the off-screen graphics object
    gg.setColor( getBackground() );
    gg.fillRect( 0, 0, width, height );
    gg.setColor( getForeground() );

    // Draw to the off-screen graphics object
    paint( gg );
    drawClock(gg);

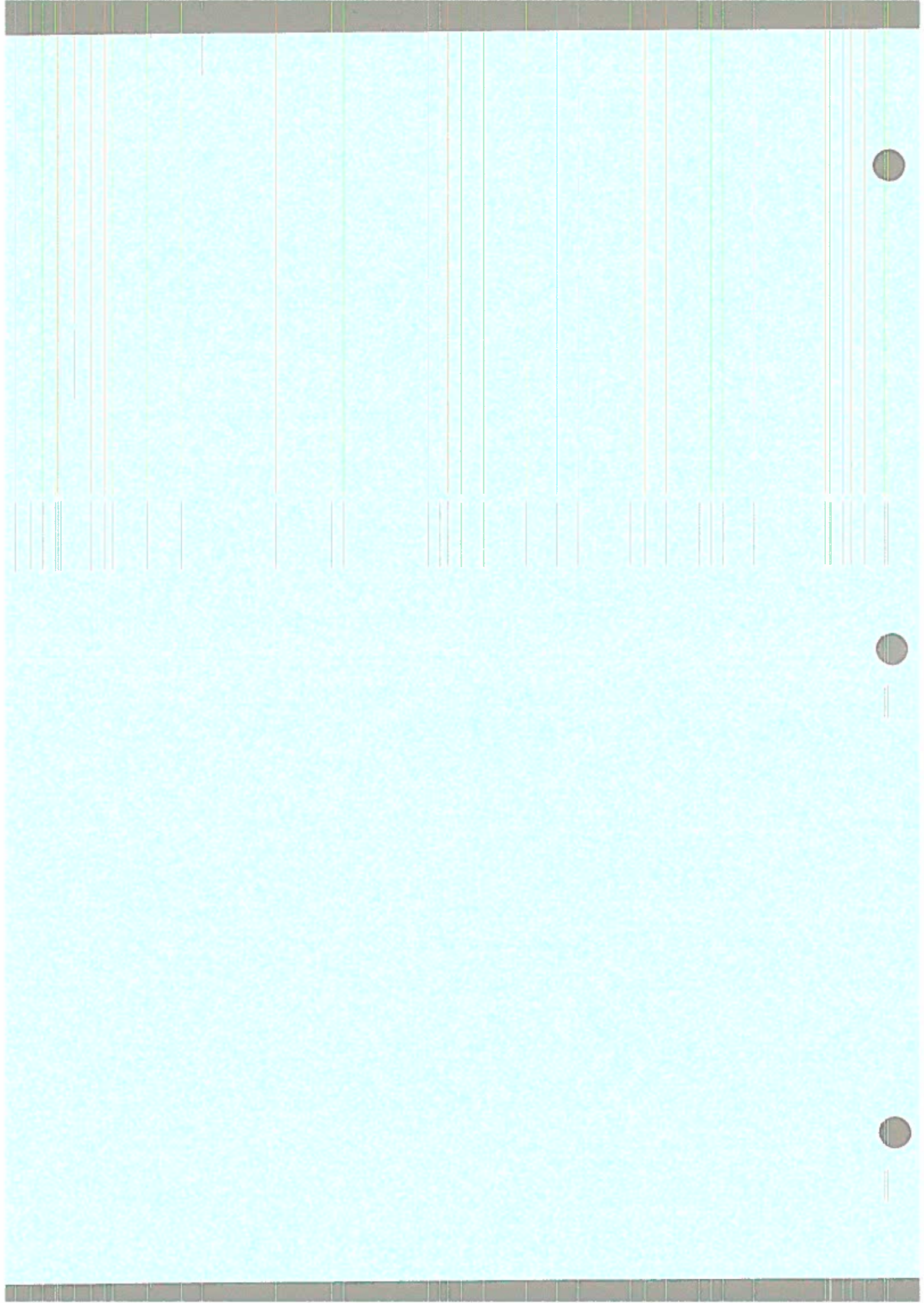
    // We don't need this Graphics object anymore
    gg.dispose();

    // Finally, we transfer the newly-drawn stuff right to the
    // screen
    g.drawImage( offscreen, 0, 0, null );
}
```

```
// -----
```



```
/*----- defines.h -----*/  
  
filename: defines.h  
  
Steven M. Garbeil  
Coins 315.  
2/1/2006  
  
description: defines for this lab. annoying. had to separate them out.  
  
defines for the stack worksheet  
  
-----*/  
#ifndef DEFINES_H  
#define DEFINES_H  
  
#define MAX 255  
  
// LEDIT FSM ///////////////////////////////////////  
  
#define INIT                0  
//#define ENTRY             1 // not really a menu  
#define COMMAND            2  
#define ENTER              3  
#define QUIT               4  
  
#define INSERT              0 // commands supported  
#define APPEND              1  
#define LIST               2  
#define DELETE             3  
#define EXIT               4  
#define ENTRY              5  
  
- static const char* CommandTickImage[6] = {"Insert","Append","List","Delete","Exit","Ent  
  
#define NOMODS             0 // command modifiers  
#define ONEMOD             1  
#define TWOMOD             2  
  
// COLORS ///////////////////////////  
  
- typedef unsigned int UINT;  
  
#endif
```

```

/*
  filename: list.h
  class: COINS 415
  instructor: YU-Ju Lin, PH.D
  assignment: HW3
  student: Steven Garbeil, Peasant
  description: this is the supporting file for HW3
  pretty much used the framework from the book by Adam Drozdek. With some many needed mods

  comments: Shoulda put this one out BEFORE the last test. the book is awful. Just thought
*/
#endif listSERVER
#define listSERVER

```

```

class list {
private:
  • node *head, *tail, *current;
  • int lsize;           // number of items in the list
  • int count;           // this is a temporary value to pass in
public:
  list() {
    head = tail = current = NULL; // create the list and make it empty
    lsize = 0;
    count = 0;
  }
  ~list();

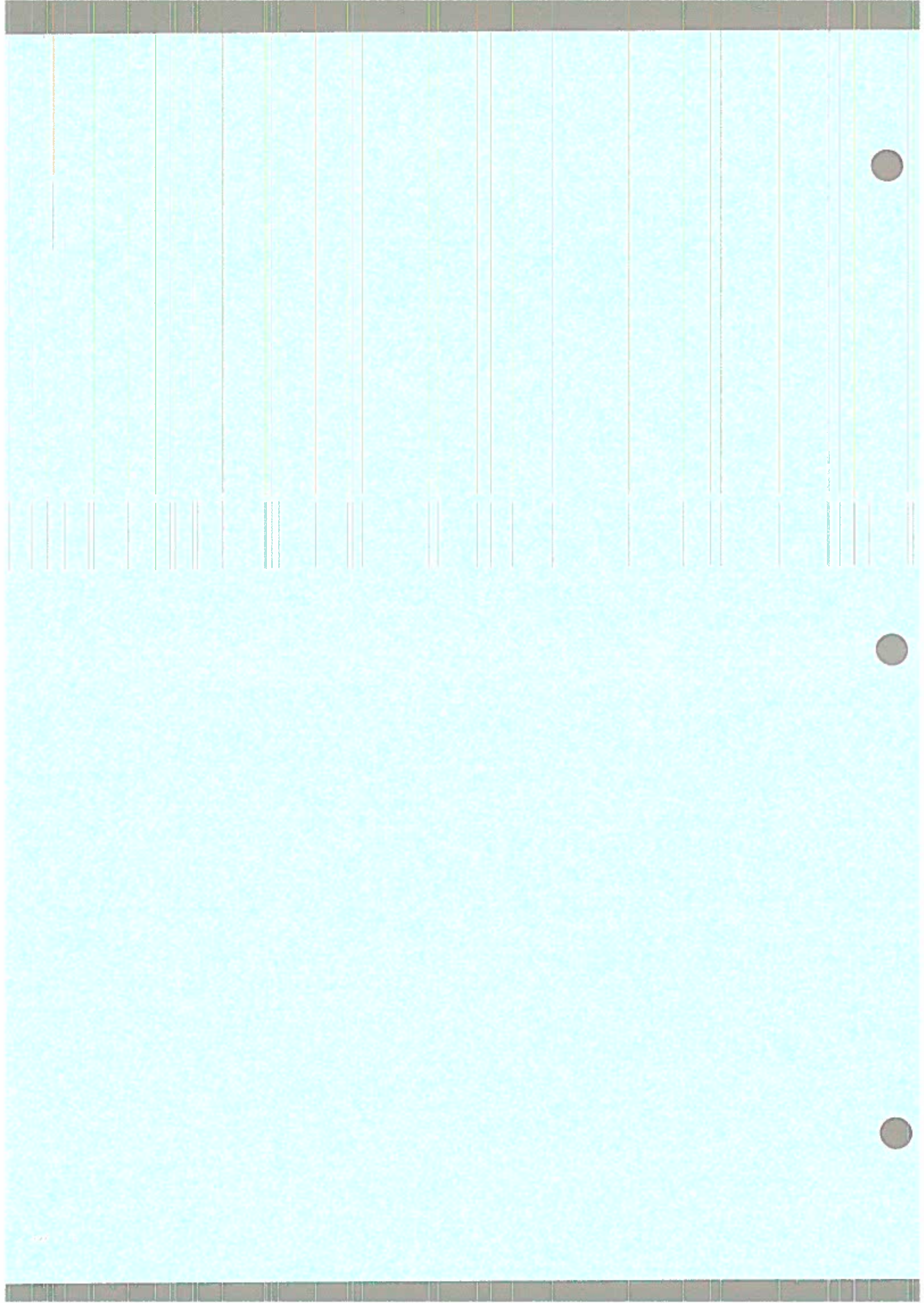
  bool isEmpty(void) {
    return (head == NULL);
  }

  • void reindex(void); // this also recalculates the size of the list
  • bool addToHead(std::string line); // adds a node to the head of the list
  • bool addToTail(std::string line); // adds a node to the tail of the list
  • std::string deleteFromHead(void); // delete the head and return its info;
  • std::string deleteFromTail(void); // delete the tail and return its info;
  • int deletenode(string); // returns the former index
  • int deletenode(node * n); // deletes by a node
  • string deletenode(int index); // returns the former string
  • int isInList(string line);
  • void printAll() const;
  • void reverse(void);

  void sethead(node * p) {
    head = p;
  }; // end set head

  node * getHead() {
    return head;
  }

```




```

    }; // end getHead

    void setTail(node * p) {
        tail = p;
    }; // end setTail

    node * getTail() {
        return tail;
    };

    int getsize(void){
        return lsize;
    };

    • void printCurrent(int index); // walks to an index then prints the contents

    • node * walk ( int index);           //untested
    • node * walkbefore(int index);       // untested
    • bool insert(int index, string line); //untested
    • bool insertbefore(int index, string line); //untested

    • friend bool operator==(list & lhs, list & rhs); // works! tested!
    • friend list & operator+(list & lhs, list & rhs); // works! tested but not completely
    //list & operator=(list & rhs); // this one needs some work!
};

// list & operator=(const list & lhs,const list & lhs); // rhs stands for right hand side
#endif

```



```

/*
filename: d:\home\sgarbeil\cs415\project1\list.cpp
this is the file I compile.
class: COINS 415 Advanced Algorithms
instructor: Yu-Ju Lin, Ph.d
assignment: Hw3
student: Steven Garbeil, Peasant

description: A highly modified list ADT from Dr. Drozdek. Modified
beyond recognition. parts taken from data structures code. it's
pretty inconsistent due to my experimentint different methods of
access.
note: the indexing is a recent addition and might need some work.

comments:

*/

//***** list.cpp *****
#define WIN32_LEAN_AND_MEAN
#define VC_ULTRALEAN

#include <iostream>
#include <string>

using namespace std;

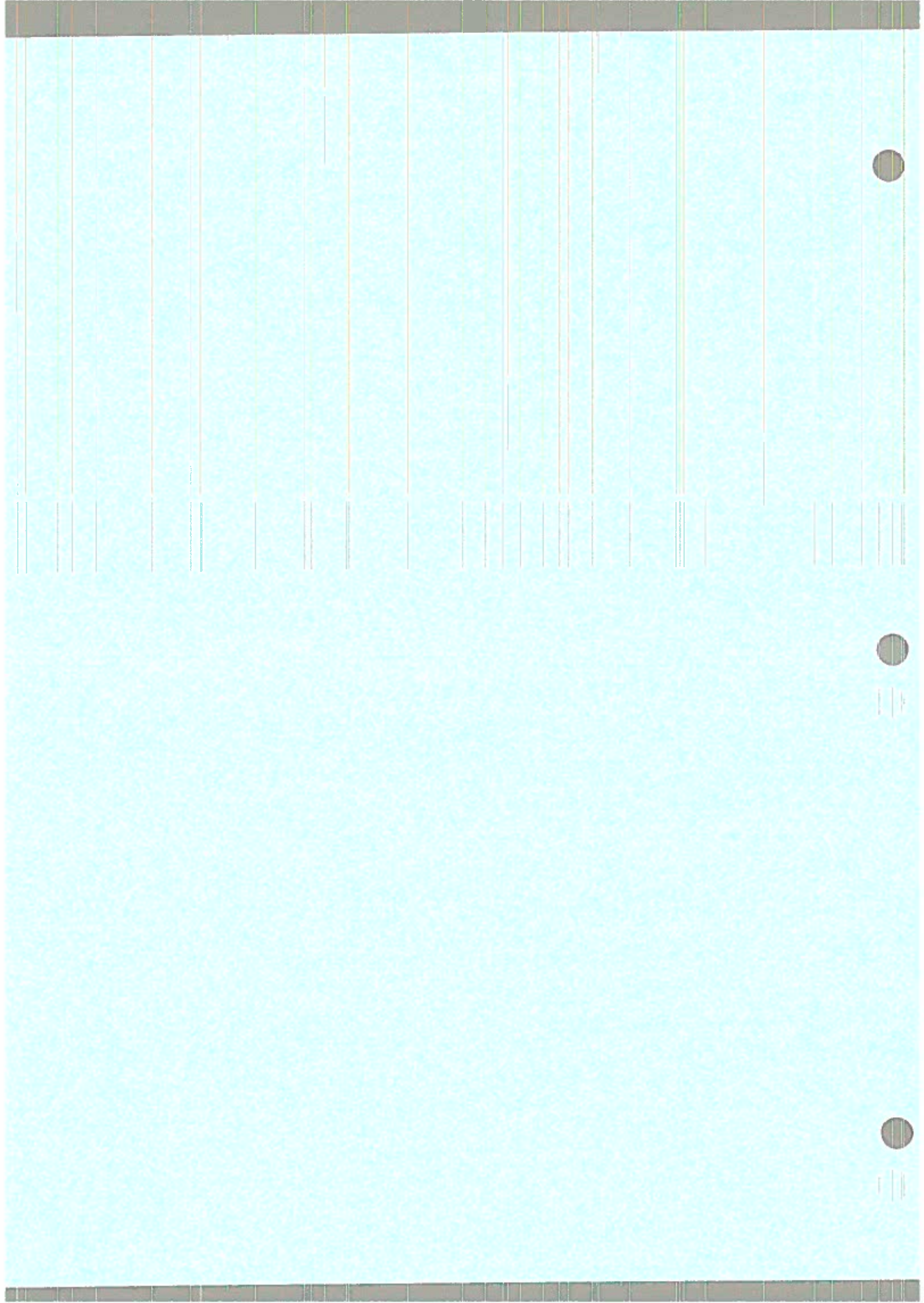
#include "node.h"
#include "list.h"

list::~list() {
    for (node *p; !isEmpty(); ) {
        p = head->next;
        delete head;
        head = p;
    }
}

// list::reindex(void) -- renumbers the

void list::reindex(void) {
    if (!isEmpty()) {
        count = 0; // first record is number 1
        current = head; // start at the top
        while (current != NULL) {
            current->index = count;
            count++; // increment the list
            current=current->next; // walk down the list
        } // end while
        current = head; // reset current
        lsize = count; // set size.
    }
}

```

```

    else {
        lsize = 0;
    }
}; // end reindex

```

```

bool list::addToHead(string line) {

    bool good = false ;
    lsize++;
    node * temp = new node(line, head);
    if (isEmpty()) {
        head = temp;
        tail = head;
        head->index = 0;
        return good;
    }
    else
        if (head == tail) { // one node
            head = temp;
            head->next = tail;
            head->index = 0;
            tail->index = 1;
            return good;
        }
        else {
            temp->next = head;
            head = temp;
            reindex();
            return good;
        }
};

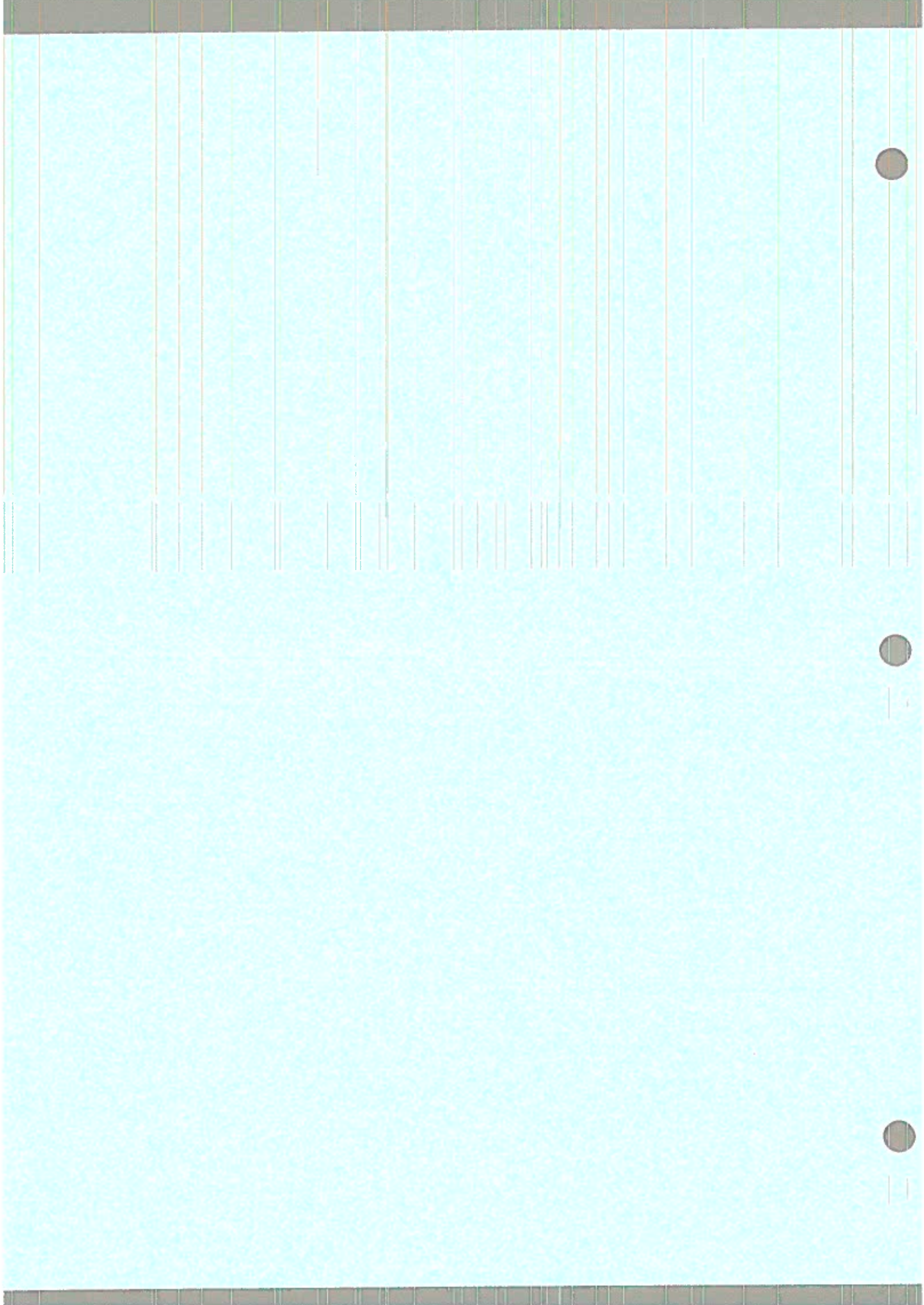
```

```

bool list::addToTail(std::string line) {

    // cout << "tailing:" << lsize << ":" << line << endl;
    node * temp = new node(line, NULL);
    lsize++;
    bool good = false ;
    if (isEmpty()){ // if no nodes on the list, add one
        head = temp;
        tail = head;
        good = true;
    }
    else {
        tail->next = temp;
        tail = tail->next;
        good = true;
        // cout << "still tailing:" << lsize << endl;
        reindex();
        // cout << "yet still tailing:" << lsize << endl;
    }
    return good;
};

```



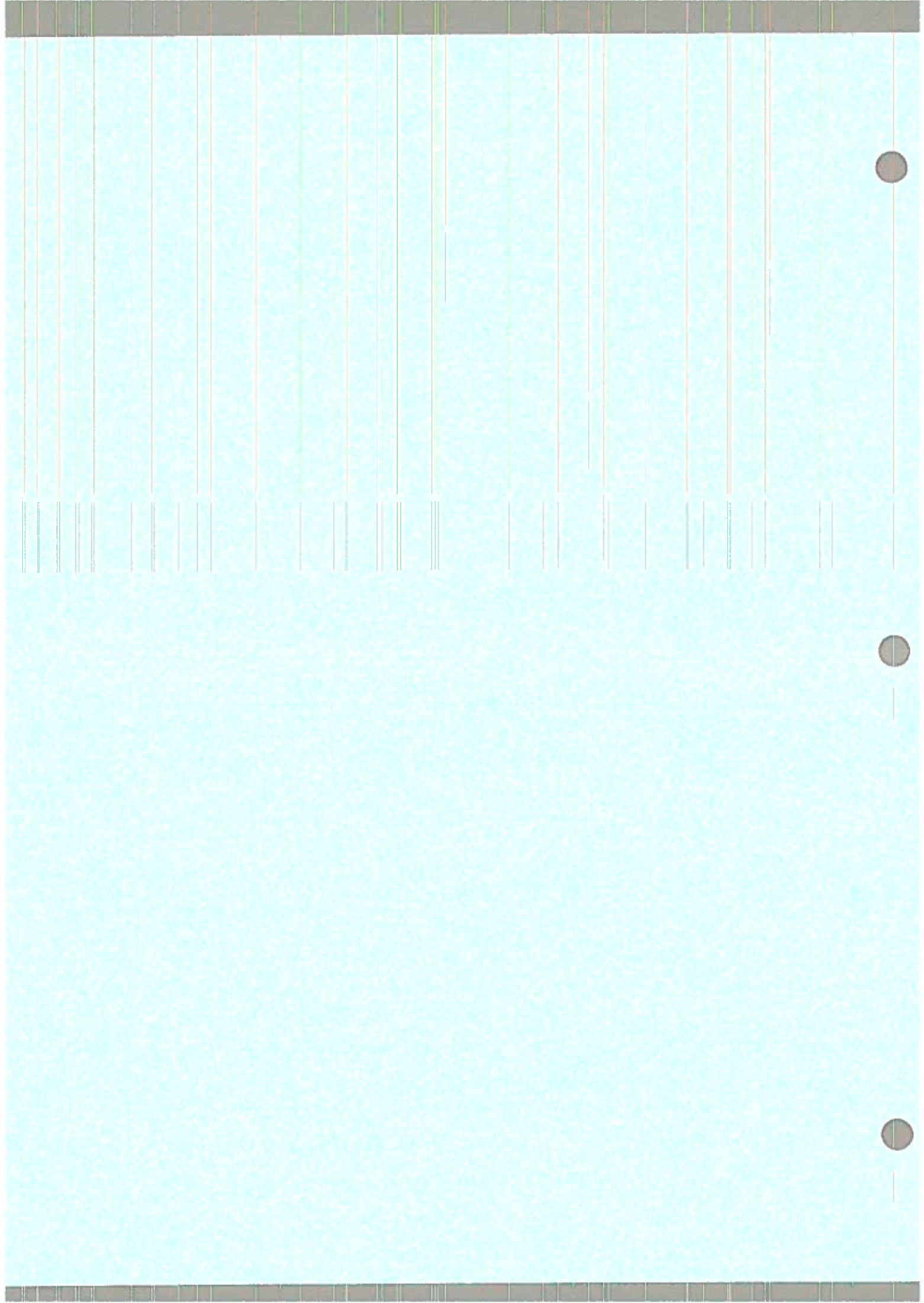

```

string list::deleteFromHead() {
    //ccox    lsize--;
    string data = head->line;
    node *temp = head;
    if (head == tail) {    // if only one node on the list; // ccox
        head = tail = 0;
        lsize = 0;    // ccox
    }
    else {    //ccox
        head = head->next;
        lsize--;    //ccox
    }    //ccox
    if (temp != 0) {    //ccox
        delete temp;
    }    //ccox
    return data;
};    // end delete from head

string list::deleteFromTail() {
    lsize--;
    string line = tail->line;
    if (head == tail) {    // if only one node on the list;
        delete head;
        head = tail = 0;
    }
    else {    // if more than one node in the list,
        node *tmp;    // find the predecessor of tail;
        for (tmp = head; tmp->next != tail; tmp = tmp->next);
        delete tail;
        tail = tmp;    // the predecessor of tail becomes tail;
        tail->next = 0;
    }
    return line;
}

int list::deletenode(node * n) {
    node * parent = head;    // set the head pointer at the beginning
    int i = n->index;
    if (isEmpty()) {
        return 0;
    }
    else if (head == n) {    // head node
        deleteFromHead();
    }
    else if (tail == n) {    // tail node
        deleteFromTail();
    }
    else {
        while (parent->next != n) {
            parent = parent->next;
        }
    }
}

```



```

    parent->next = n->next;    // take n out of the chain
    delete n;                // delete it
}
reindex();
return i;
} // end delete node

```

// deletes the found string and passes back the deleted index
 // which is fairly meaning less since it then reindexes the list
 // I wrote this one in data structures! modified it here

```

int list::deletenode(string data) {
    lsize--;
    int i = 0;
    if (head != 0) // if non-empty list;
        if (head == tail && data == head->line) { // if only one
            if (head->line == data) {
                i = head->index;
                delete head; // node on the list;
                head = tail = 0;
            }
        }
        else
            if (data == head->line) { // if more than one node on the list
                node *tmp = head;
                head = head->next;
                delete tmp; // and old head is deleted;
            }
            else { // if more than one node in the list
                node *pred, *tmp;
                for (pred = head, tmp = head->next; // and a non-head node
                    tmp != 0 && !(tmp->line == data); // is deleted;
                    pred = pred->next, tmp = tmp->next);
                if (tmp != 0) {
                    pred->next = tmp->next;
                    if (tmp == tail)
                        tail = pred;
                    i = tmp->index;
                    delete tmp;
                }
            }
    reindex();
    return i;
}; //end deletenode(string)

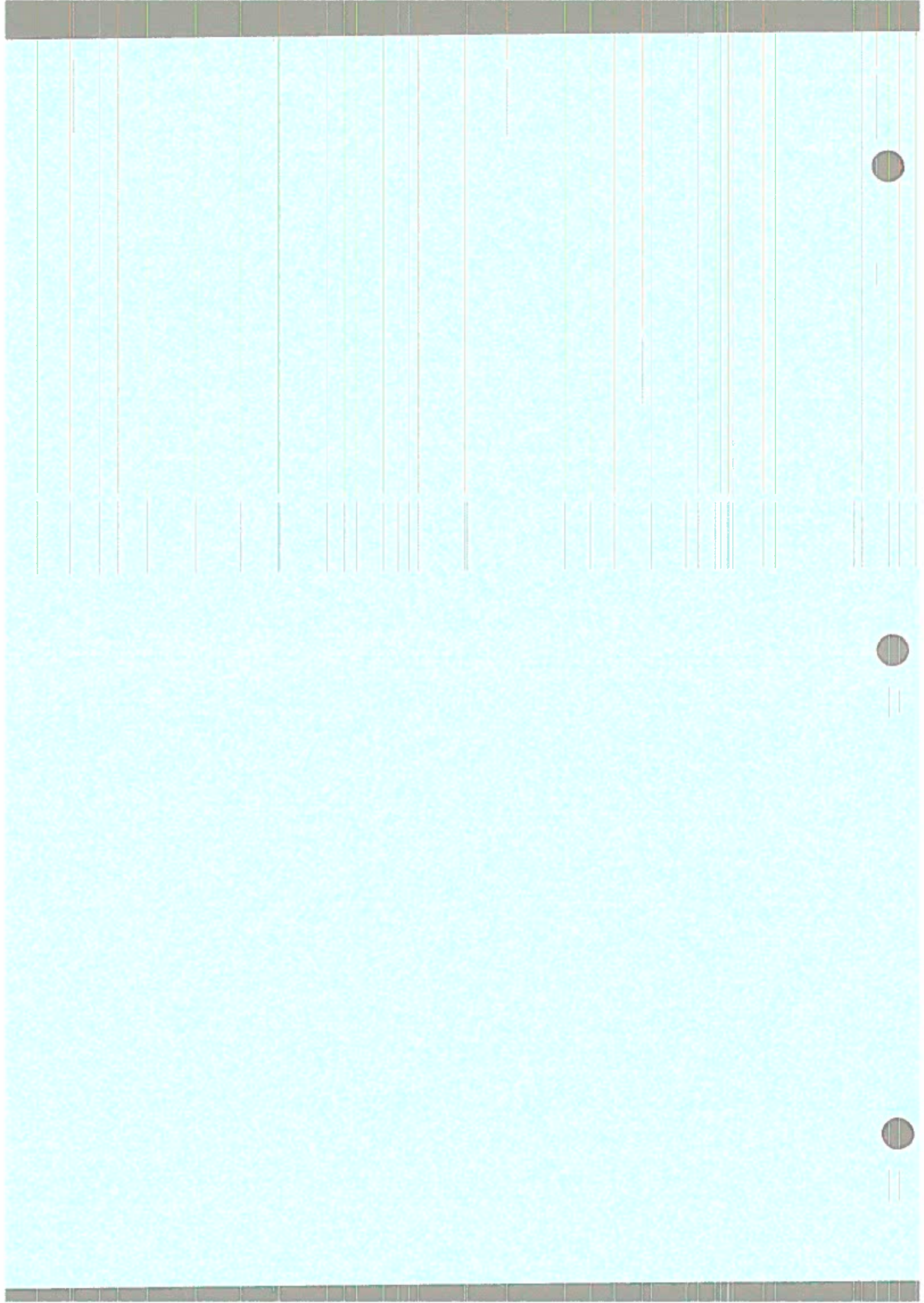
```

// deletenode overloaded for index. amazing the difference in coding technique.
 // especially when you don't have an overly clever lab manual dictating your code.
 // the special cases are handled in the delete routines.

```

string list::deletenode(int index) {

```

```

string s;
node * n = head;
// assume a valid index
if (isEmpty()) { // if the list is empty
    s = "\0";
}
else if (index >= lsize || index < 0) { // if the index is out of range
    cerr << "Invalid Index: " << index << endl;
    s = "\0";
}
else if (index == 0) { // delete the head
    s = deleteFromHead();
}
else if (index == 1 && lsize == 2) { // ccox
    s = deleteFromTail(); //ccox
} //ccox
else if (index == (lsize - 1)) {
    s = deleteFromTail();
}
else {
    while (n->index != index) {
        n = n->next;
    }
    deletenode(n);
}
return s;
};

```

// walks the list searching for a match. if it finds one, it returns
 // the index of the node. this used to just say yea or nay. dumb.

```

int list::isInList(string line) {
    int i = -1;
    while (current != NULL) {
        if (current->line == line) {
            i = current->index;
            break;
        }
        else
            current = current->next;
    } // end while
    return i;
};

```

```

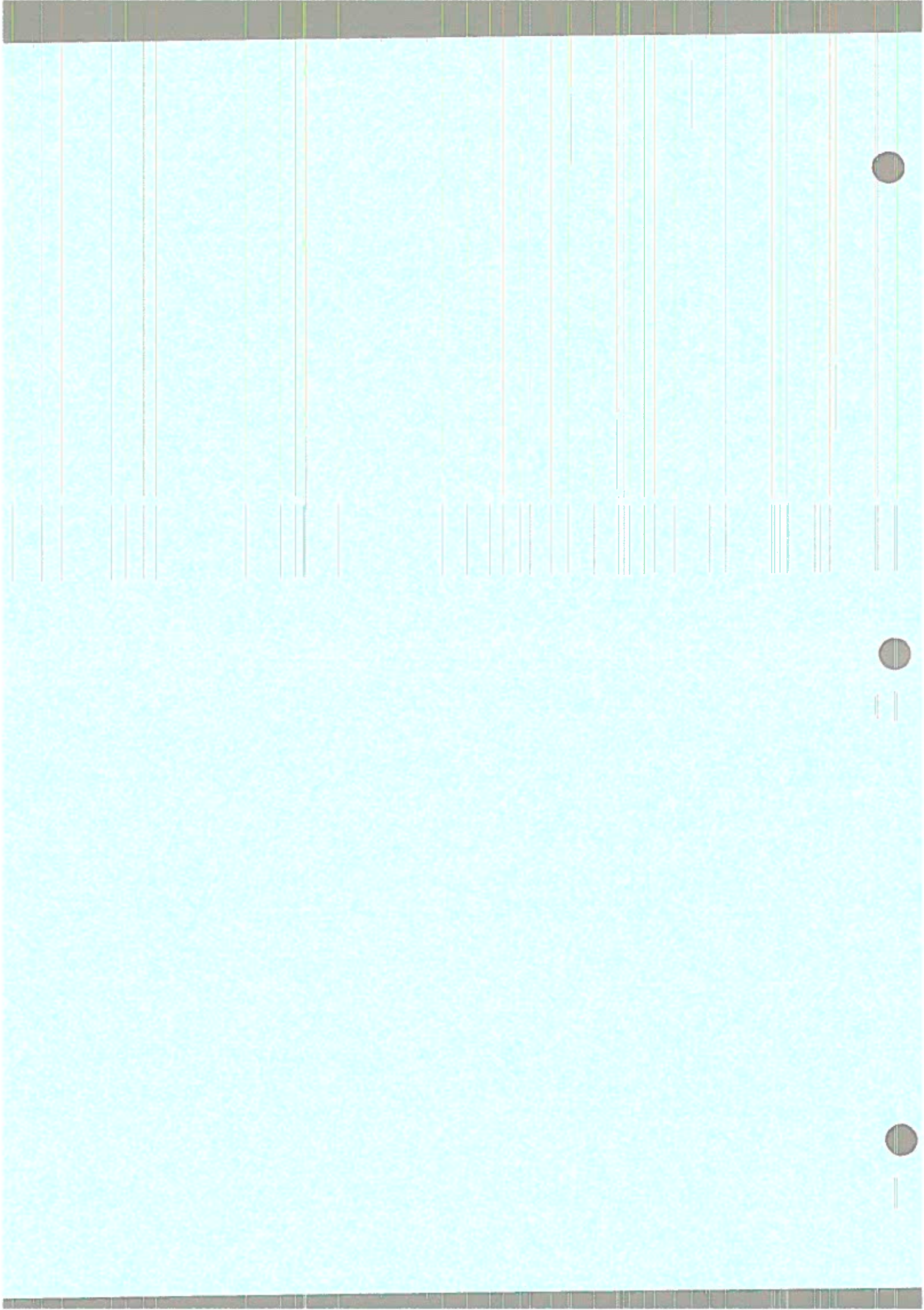
void list::printAll() const {
    for (node *tmp = head; tmp != 0; tmp = tmp->next)
        cout << tmp->line << endl;
    cout << endl;
};

```

```

void list::printCurrent(int index) {
    current = walk(index);
}

```




```

    cout << current->line << endl;
};

```

```

void list::reverse(void) { // the head is passed in
    node * a = head;
    tail = head; // set the new tail
    node * parent = NULL;
    while (a != NULL) {
        node * atemp = a->next; // save the next pointer
        a->next = parent; // change the next to the parent
        parent = a; // move up the parent
        a = atemp;
    } // end while
    head = parent; // a is new head pointer
}

```

```

node * list::walk(int index) {
    if (current->index > index) { // if nPtr is null or we pass the index
        cout << "for location debugging . . . (current->index~" << current->index << "~ : ";
        current = head;
        count++;
        return walk(index); //
    }
    else if (current->index == index) { // if nPtr is null or we pass the index
        cout << "for location debugging . . . (current->index~" << current->index << "~ : ";
        // node * temp= current;
        // current = head;
        return current; //
    }
    else if ((current->index < index) && (current == tail)) { // if nPtr is null or we pass the index
        cout << "for location debugging . . . ((current->index~" << current->index << "~ : ";
        current = head;
        return tail; //
    }
    else {
        cout << "for location debugging . . . else" << endl;
        current = current->next;
        count++;
        return walk(index); // ccox
    }
}; // end walk

```

```

node * list::walkbefore (int index) {
    return walk(index - 1);
}; // end walk

```

```

void testerror(void) {
};

```



```
bool list::insertbefore(int index, string line){
```

```
    return insert(index,line);
```

```
} // not implemented
```

```
bool list::insert(int index, string line) { // inserts before an index
```

```
    bool good = false;
```

```
    // index wrapper!
```

```
    if (index < 0 || index > lsize+1){ // can only add one to the list negatives are 1
        cerr << "Illegal Index!" << endl; // die, die, die.
    }
```

```
    else
```

```
        if (index == 0) {
```

```
            good = addToHead(line);
```

```
        } // end if index = 0
```

```
        else // if not the head do some other stuff.
```

```
            if (index == lsize) {
```

```
                good = addToTail(line);
```

```
            }
```

```
            else { // not head, or tail , or empty
```

```
                current = walkbefore(index); // walk to the one before.
```

```
                current->next = new node(line,current->next); //returns pointer to cre
```

```
                lsize++;
```

```
                // add one to size not really necessary
```

```
                good = true;
```

```
                // error
```

```
            } // end else not empty list or head of the list.
```

```
        } // end else
```

```
        reindex();
```

```
        return good;
```

```
}; // end insert .. adds elements.
```

```
// return true if contents of two linked lists are the same.
```

```
// this is only true if they 1. exist, 2. have exactly the same data...
```

```
bool operator==(list & lhs, list & rhs) {
```

```
    int i = 0;
```

```
    int lhslen = 0, rhslen = 0; // length variables
```

```
    node * p = lhs.head;
```

```
    // temp pointer for the lists
```

```
    node * q = rhs.head;
```

```
    if (lhs.isEmpty() || rhs.isEmpty())
```

```
        return false;
```

```
    else {
```

```
        while (p) { // find the len for the left
```

```
            lhslen++;
```

```
            p=p->next;
```

```
        } ; // end while
```

```
        while (q) { // find the len for the right
```

```
            rhslen++;
```

```
            q = q->next;
```

```
        } ; // end while
```

```
        if (lhslen != rhslen)
```

```
            return false; // we only continue if the lists are the same len
```

```
        else {
```



```

    p = lhs.head;                // reset the head pointers
    q = rhs.head;
    for(i=0; i < rhslen ; i++){
        cout<< "(" << p->line << "," << q->line << ")" << endl;
        if (p->line != q->line) {
            return false;        // if the data doesn't match bomb
        }
        else {                    // else roll on through
            p=p->next;
            q=q->next;
        } //;
    }
    return true;                  // if we get to here, this should be a go
} // end else
};

```

// adds to lists together using the same rules as standard addition.

```

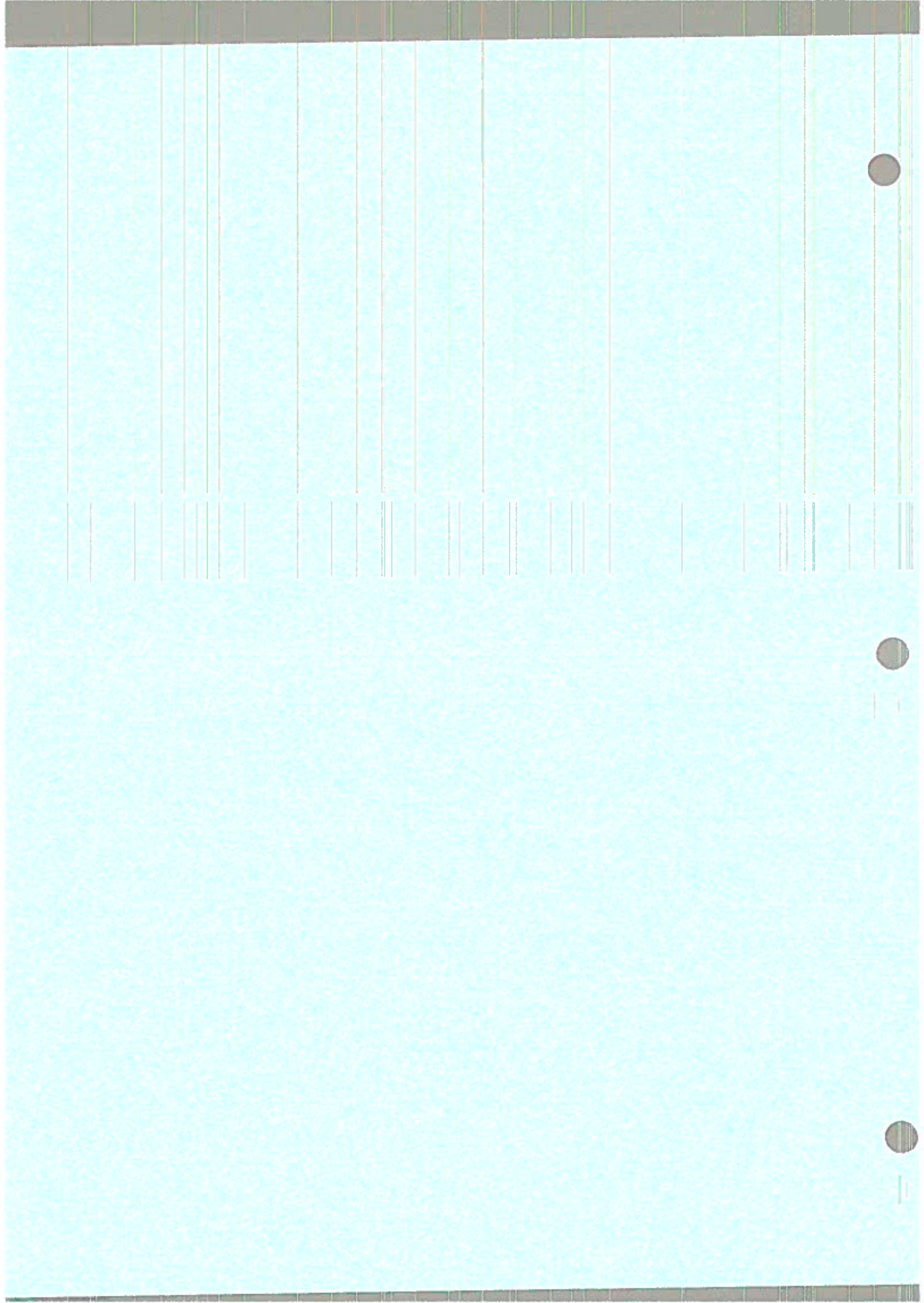
list & operator+(list & lhs, list & rhs) {
    if (lhs.isEmpty()) {
        if (rhs.isEmpty()) {
            return lhs;          // if both lists are empty return the empty list
        }
        else {
            return rhs;          // if the first list is return the other
        }
    }
    else {
        if (rhs.isEmpty()) {    // if
            return lhs;
        }
        else {
            lhs.tail->next = rhs.head;    // add the lists
            lhs.tail = rhs.tail;         //
            return rhs;
        }
    } // end else
}; // end operator

```

```

// simple assignment. this takes the left hand side and makes it equal the right.
// unary operation only requires one operator. really q
// list & operator=( list & rhs) {
// if (&rhs == this) {
//     return *this;            // handle the self assignment without a bomb
// }
// return rhs;                  // i'm not sure why this is such a big deal
//}; //end operator=
//

```




```
/*
  filename: node.h
  class: COINS 415
  instructor: YU-Ju Lin, PH.D
  assignment: HW3
  student: Steven Garbeil, Peasant
  description: this is the supporting file for HW singly-linked list class to store string
  comments: Shoulda put this one out BEFORE the last test.
*/

#ifndef _LIST
#define _LIST

class node {
public:
  - string line;
  - node * next;
  - int    index;           //
  // constructor

  node(string el = 0, node *ptr = NULL) { // this is a copy constructor.
    line = el;
    next = ptr;
  }
};

#endif
```

